

GraphRAG 实践应用白皮书

目录

第一章：知识图谱

- 知识图谱概述
- 知识的表示、存储和查询、抽取

第二章：GraphRAG 整体概述

第二章：GraphRAG 实践过程中面临的挑战

第三章：GraphRAG 的图形化展示

第四章：GraphRAG with agentic rag

- 盲点和挑战
- 应对方案

第五章：GraphRAG 的最佳实践

- 应用场景分析
- 其他案例实践

第六章：未来展望

第一章：知识图谱

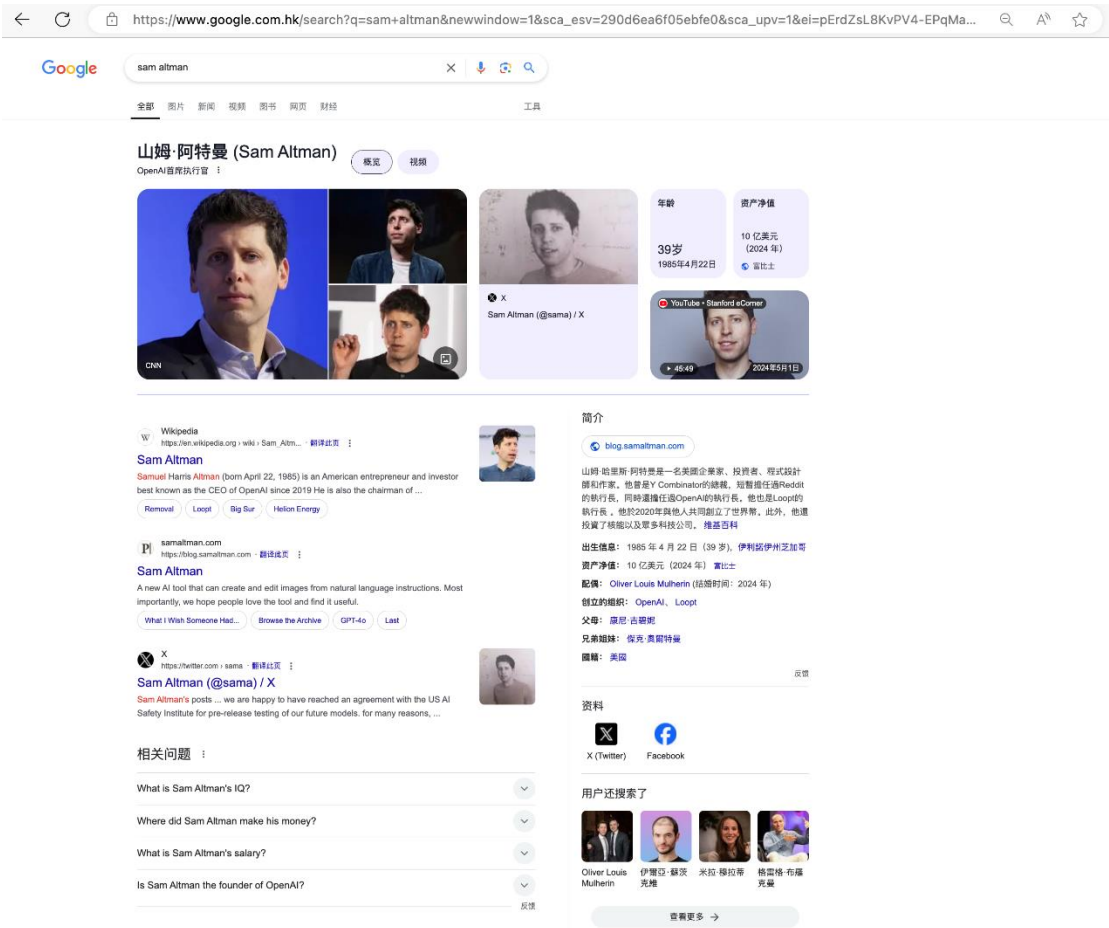
知识图谱是结构化知识表示的一种形式，它将知识组织成一个多关系图，其中节点表示实体，边表示实体之间的关系。知识图谱可以表示实体之间的语义关系，帮助机器理解和推理自然语言文本。

支持用户按主题而不是字符串检索，真正实现语义检索。基于知识图谱的搜索引擎，能够以图形方式向用户反馈结构化的知识，不必浏览大量文档即能准确定位和深度获取知识。

1. 知识图谱概述

1.1. 知识图谱的发展

2012 年，谷歌正式提出了知识图谱（ Knowledge Graph）的概念，旨在实现更智能的搜索引擎，并且于 2013 年以后开始在学术界和业界普及。下图是 Goolge 搜索中的结果，不仅仅给出了 pagerank，还给出了知识图谱的搜索结果。



目前，随着智能信息服务应用的不断发展，知识图谱已被广泛应用于智能搜索、智能问答、个性化推荐、情报分析、反欺诈等领域。另外，通过知识图谱能够将 Web 上的信息、数据以及链接关系聚集为知识，使信息资源更易于计算、理解以及评价，并且形成一套 Web 语义知识库。知识图谱以其强大的语义处理能力与开放互联能力，可为万

维网上的知识互联奠定扎实的基础，使 Web 3.0 提出的“知识之网”愿景成为了可能。

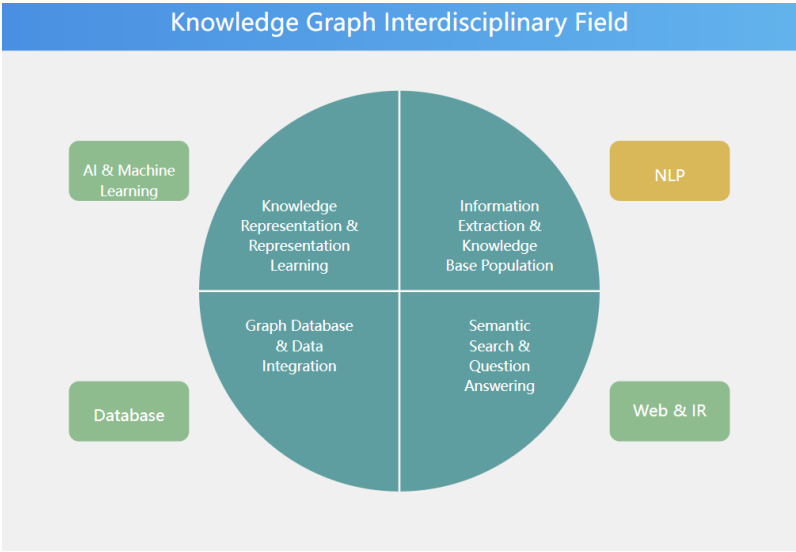
要追溯知识图谱的起源，我们可以回到 20 世纪 60 年代提出的语义网络（Semantic Networks）。

以下是其发展脉络：



1.2. 知识图谱覆盖的领域

知识图谱的覆盖了知识图谱的表示、存储、获取、推理、融合、问答、分析等七大方面。同时，它还包含了与其他学科领域的交叉融合，主要可以分为以下四个方面：



1.3. 知识图谱的组成

1.3.1 实体

实体是知识图谱中的基本单元，通常是名词或概念。实体可以是具体的对象（如人、地点、物品）或抽象的概念（如概念、类别）。

1.3.2 关系

关系是知识图谱中实体之间的连接，表示实体之间的语义关系。关系可以是二元的（表示两个实体之间的关系）或

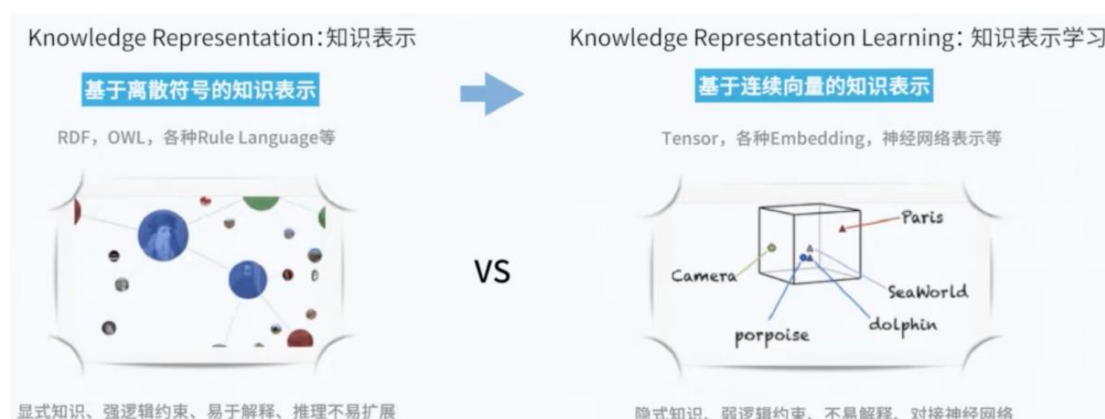
多元的（表示多个实体之间的关系）。

2. 知识图谱中的知识表示

知识表示的核心是通过某种形式来描述、组织和存储知识，便于机器理解与推理。知识表示主要用途如下：



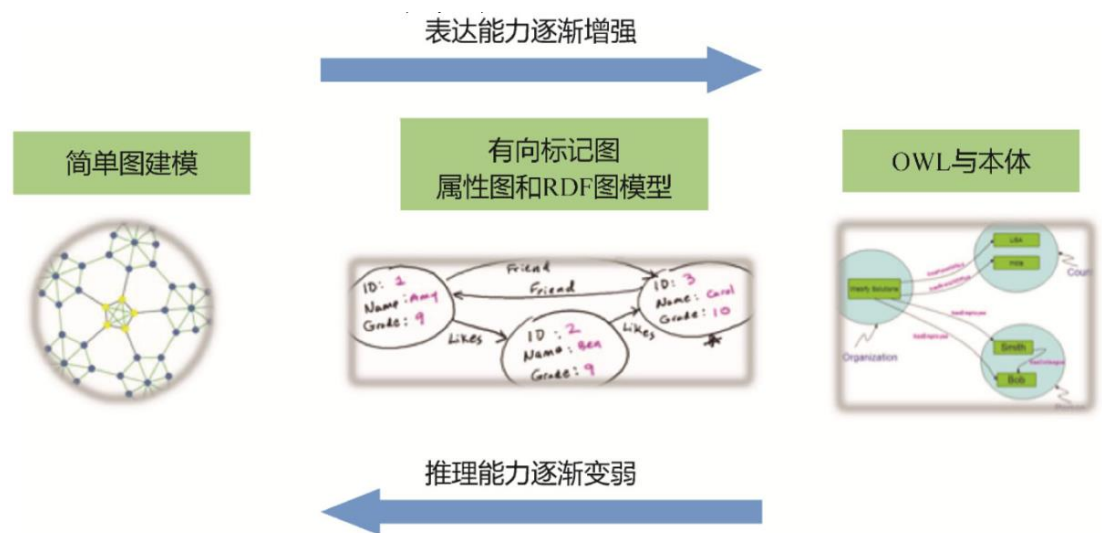
而知识表示主要表示方法有：



2.1. 知识图谱的符号表示方法

知识图谱通过图的形式来描述和表达知识，能够构建更复杂的实体关系模型。但在知识图谱的实际实践中，不同的应用场景会对知识的建模采用不同表达能力的图表示方法。

知识图谱表示应用最多的是有向标记图（Directed Labelled Graph）。最常用的两种有向标记图模型，一种叫属性图（Property Graph），另外一种为 RDF 图模型。但有向标记图的表达能力依然是有限的，在很多专业领域，如医学本体构建等，需要更为复杂的关系语义的表示，例如描述对称关系、自反关系、传递关系等，这就需要用到 OWL 等本体描述语言。知识图谱领域三种最常用的知识表示方法——属性图、RDF 图模型和 OWL 本体语言。



2.1.1 属性图

属性图是图数据库 Neo4J 实现的图结构表示模型，在工业界有广泛应用。属性图的优点是表达方式非常灵活，例如，它允许为边增加属性，非常便于表示多元关系。属性图的存储充分利用图的结构进行优化，因而在查询计算方面具有较大优势。属性图的缺点是缺乏工业标准规范的支持，由于不关注更深层的语义表达，也不支持符号逻辑推理。

2.1.2 RDF 图模型

RDF 是国际万维网联盟 W3C 推动的面向 Web 的语义数据标准，所以 RDF 本身的定位首先是数据交换标准规范，而非存储模型。RDF 的基本组成单元是三元组，即(s,p,o)，例如，可以用一条三元组来描述<浙江大学, 位于, 杭州>。一条三元组代表关于客观世界的逻辑描述或客观事实。多个三元组头尾相互连接，就形成了一个 RDF 图。本质上，RDF 图也是一个有向标记图。

RDF 还提供了基础的表达构件用于定义类、属性等 Schema 层的术语。例如，domain、range 用于定义某个关系的头尾节点类型，subClassOf 和 subPropertyOf 用于定义类及属性之间的层次关系等。有了 RDFS，已经可以实现最简单的符号推理。例如：如果将谷歌定义为一家人工智能公司，同时也知道人工智能公司是高科技公司，就可以推理得出谷歌也是一家高科技公司。这类简单的推理已经可以减轻数据维护的负担，因为很多查询结果可以通过推理间接得出，而不用对所有结果进行全量存储。

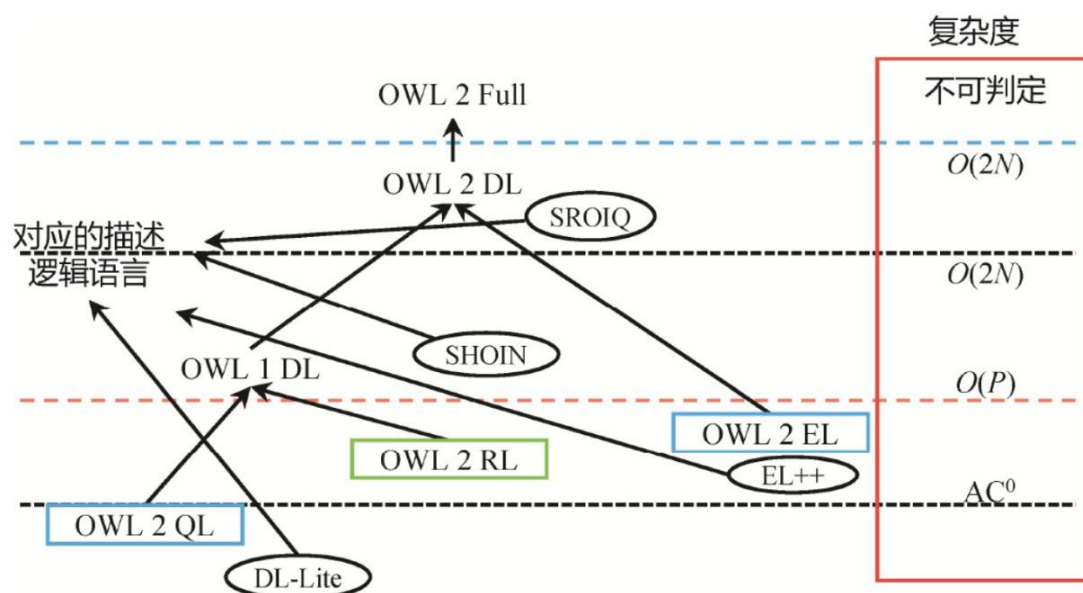


2.1.3 OWL 本体语言

RDF+RDFS 的表达能力是非常有限的。在实际应用中，需要定义更为复杂的概念，刻画更为复杂的概念关系。这就需要用到 OWL 等本体表达语言。本体（Ontology）原来是一个哲学术语，后来被人工智能的研究人员作为知识表示研究的对象引入计算机领域。本体最常用的逻辑表达语言即描述逻辑（Description Logic）。

OWL 首先可以被看作 RDF Schema 的扩展。OWL 在 RDF 的基础上增加了更多的语义表达构件。例如，通过多个类组合定义更加复杂的类；刻画关系的一对多、多对一、多对多等关系基数（Cardinality）约束；定义常用的全称量词和存在量词；定义互反关系、传递关系、自反关系、函数关系等更加复杂的关系语义等。

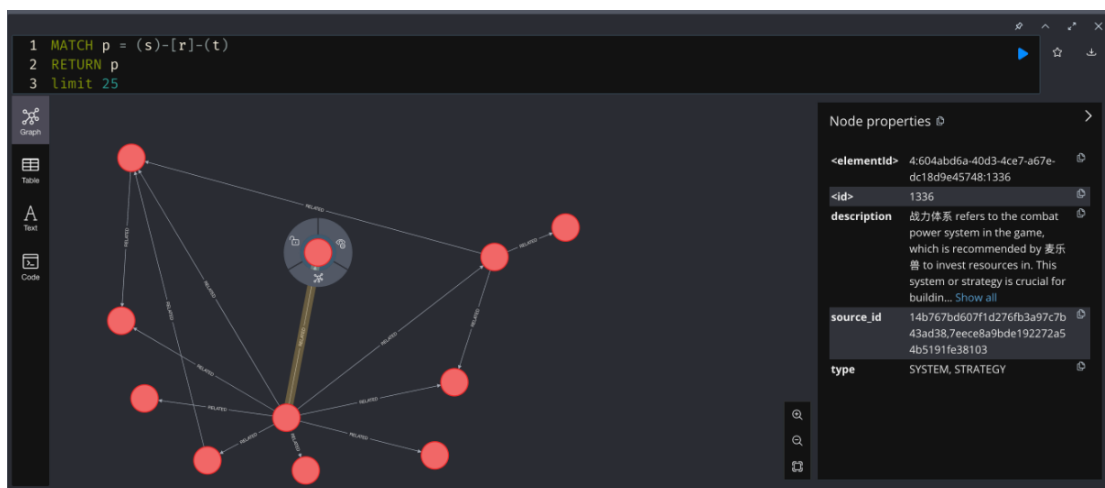
OWL 拥有众多的表达构件。OWL 实际上有很多语言家族，不同的语言家族代表不同的表达构件的组合。不同的组合对应不同的推理计算复杂度以及它们所适用的场景。OWL-QL 表示查询语言的意思，专为基于大规模的查询设计的子语言；OWL 2 RL 在扩展 RDFS 表达能力的同时，保持了较低的复杂度；OWL 2 EL 专为概念术语描述、推理而设计，在生物医疗领域广泛应用，如临床医疗术语本体 SNOMED CT 等。



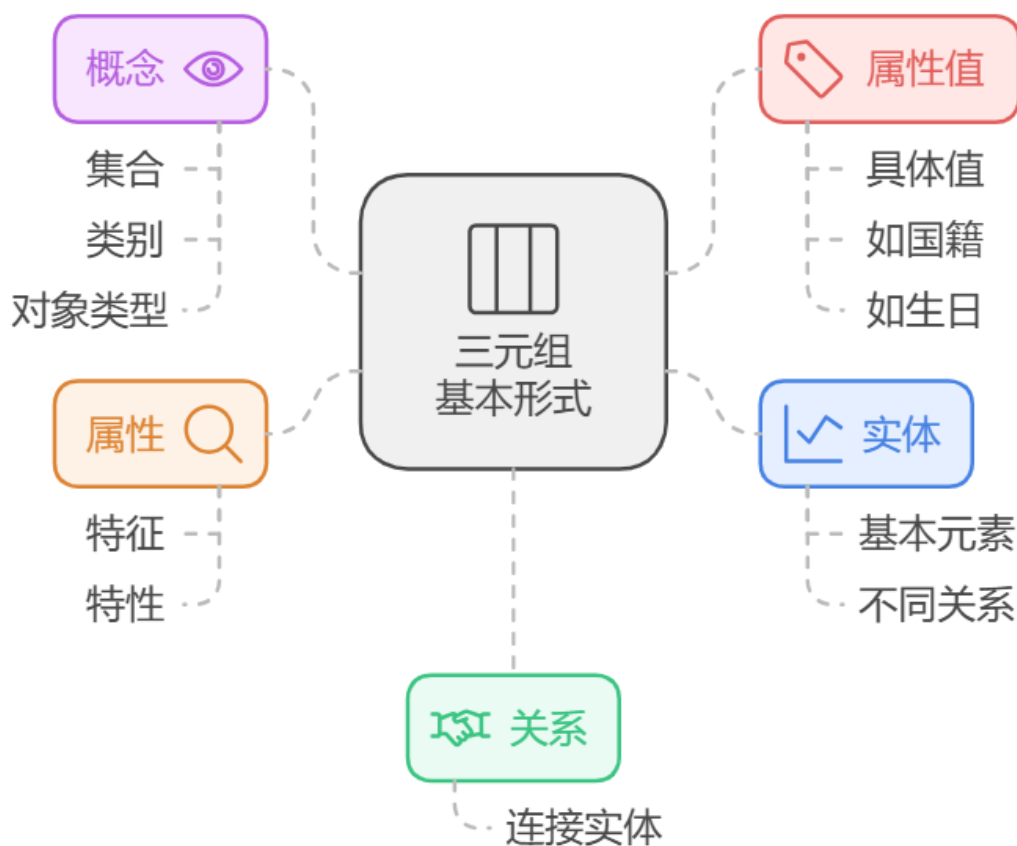
2.1. 最常用的知识表示方法--属性图(neo4j 的实现)

而在知识表示中，知识图谱是一种知识库，其中的数据通过图结构的数据模型或拓扑整合而成。知识图谱通常被用来存储彼此之间具有相互联系的实体。

三元组是知识图谱的一种通用表示方式，即 $G = (E, R, S)$ ，其中 E 是知识库中的实体， R 是知识库中的关系， S 代表知识库中的三元组。



三元组的基本形式主要包括实体关系、实体和概念、属性、属性值等。



- 实体是知识图谱中的最基本元素，不同的实体间存在不同的关系。
- 概念主要指集合、类别、对象类型、事物的种类，例如人物、地理等。

- 属性主要指对象可能具有的属性、特征、特性、特点以及参数，例如国籍、生日等；属性值主要指对象指定属性的值，例如中国、1988-09-08 等。

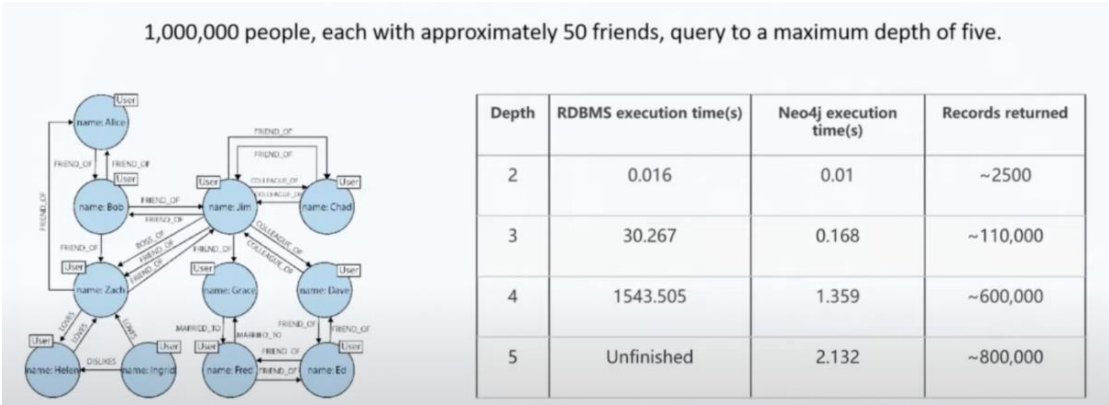
每个实体(概念的外延)可用一个全局唯一确定的 ID 来标识，每个属性-属性值对(attribute-value pair, AVP) 可用来刻画实体的内在特性，而关系可用来连接两个实体，刻画它们之间的关联。

3. 知识的存储和查询

随着数据量的爆炸式增长和数据处理需求的复杂性提升，数据库技术也在不断发展和演进。 传统的关系型数据库以其稳定的数据结构和强大的事务处理能力，在各类业务系统中占据了重要地位。 然而，随着大数据和实时分析需求的崛起，关系型数据库在某些场景下的局限性逐渐显现，图数据库脱颖而出。

3.1. 图数据库特点

- 高效处理复杂的关系：图数据库擅长管理高度连接的数据。当处理具有大量相互连接的数据时，例如社交网络或组织层次结构，图数据库提供了明显的优势。它们将数据表示为节点和边，使得对复杂关系进行建模和查询变得直观。



- 可变模式：图数据库允许不断变化的模式，这在处理动态数据模型时特别有用。与需要预定义模式的关系数据库不同，图数据库提供了灵活性，允许您在数据模型不断发展时添加新的关系类型或实体。

在需要更加深入的研究数据之间的关系时，需要更加丰富的关系语义的表达能力，除了前述 Reflexive和多跳关系、还包括传递关系 Transitive、对称关系 (非对称关系) Symmetric、反关系 (Inverse)、函数关系 (Functional)

除了关联查询能力，深层次的关系建模还将提供关联推理的能力，属性图数据库如Neo4J提供了关系模型的关联查询能力，AllegroGraph等RDF图数据库提供了更多的关联推理能力

Symmetric Property

Transitive Property

`:hasSpouse rdf:type owl:SymmetricProperty .`

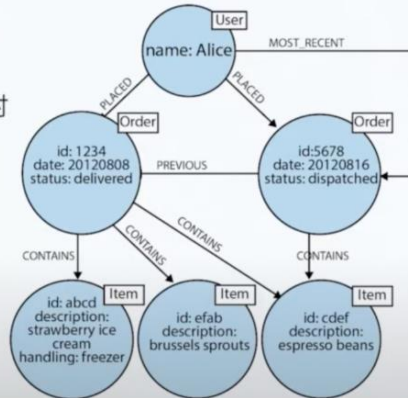
`:hasAncestor rdf:type owl:TransitiveProperty .`

📖 自然表达:图是十分自然的描述事物关系的方式,更加接近于人脑对客观事物的记忆方式

📖 易于扩展:图模型更加易于适应变化,例如在图中,临时希望获取历史订单,只需新增边即可

📖 复杂关联表达:图模型易于表达复杂关联逻辑的查询,例如在推荐系统中,希望表达复杂的推荐逻辑,例如:“all the flavors of ice cream liked by people who enjoy espresso but dislike Brussels sprouts, and who live in a particular neighborhood.”

📖 多跳优化:在处理多跳查询上,图模型有性能优势



3.2. 图数据库的分类

2.2.1 从查询语句维度分

• 支持 Cypher 查询语言的图数据库

Cypher 是一种基于声明式图查询语言,最早由 Neo4j 开发,用于查询属性图 (Property Graph)。以下是一些支持 Cypher 的数据库:

- Neo4j
 - 描述: Neo4j 是最知名的图数据库之一, Cypher 是它的原生查询语言。
 - 特点: 强调图模式的简单性和关系的高效查询,适用于复杂的关系查询。
- Memgraph
 - 描述: Memgraph 是一个实时流图数据库,支持 Cypher 查询语言。
 - 特点: 强调实时性和高性能,适合数据流应用。
- RedisGraph
 - 描述: RedisGraph 是 Redis 的模块之一,支持 Cypher 查询语言。
 - 特点: 轻量、高效,集成在 Redis 中,适合需要快速、简单图查询的场景。

• 支持 Gremlin 查询语言的图数据库

Gremlin 是 Apache TinkerPop 图处理框架的一部分,用于属性图查询。以下数据库支持 Gremlin 查询:

- JanusGraph
 - 描述: JanusGraph 是一个分布式图数据库,支持大规模图数据存储和查询,原生支持 Gremlin。
 - 特点: 适用于大规模图数据、强一致性和扩展性。
- Amazon Neptune
 - 描述: Amazon Neptune 是 AWS 的托管图数据库,支持 Gremlin 和 SPARQL。
 - 特点: 云原生设计,支持属性图 (Gremlin) 和 RDF (SPARQL),适合多模型图数据库应用。
- Azure Cosmos DB (Gremlin API)
 - 描述: Azure Cosmos DB 是微软的多模型数据库,支持 Gremlin 查询语言。
 - 特点: 提供全球分布式、高可用性的图数据库解决方案,支持多模型架构。

• 支持 SPARQL 查询语言的图数据库

SPARQL 是一种用于查询 RDF 图的标准查询语言。以下是一些支持 SPARQL 的数据库:

- Virtuoso
 - 描述: Virtuoso 是一个多用途数据库管理系统,支持 SPARQL 查询。
 - 特点: 支持大规模 RDF 数据存储,广泛用于语义网应用。
- GraphDB

- 描述: GraphDB 是一个企业级的 RDF 图数据库, 支持 SPARQL 查询。
- 特点: 强大的推理能力和高效的 SPARQL 查询, 适用于语义网和本体开发。
- Blazegraph
 - 描述: Blazegraph 是一个高性能的 RDF 图数据库, 支持 SPARQL 1.1 标准。
 - 特点: 适合大规模 RDF 数据查询, 常用于学术和研究项目。

3.2.1 基于属性图、RDF 图模型和 OWL (Web 本体语言) 分

• 基于属性图的图数据库

属性图数据库 (如 Neo4j) 更适合于有复杂关系和动态属性的数据, 主要关注图的结构和属性。

主要代表:

- Neo4j: 是最流行的图数据库之一, 使用属性图模型, 允许节点和边拥有属性。
- JanusGraph: 一个分布式图数据库, 支持属性图模型, 可以与大数据技术 (如 HBase、Cassandra) 集成。
- ArangoDB: 一个多模型数据库, 支持属性图和文档模型。

特点:

- 模型: 图由节点 (顶点)、边 (关系) 和属性 (键值对) 组成。节点和边都可以有属性。
- 查询语言: Neo4j 使用 Cypher 查询语言, ArangoDB 使用 AQL (ArangoDB 查询语言)。

2.基于 RDF 图模型的图数据库

RDF 图数据库 (如 Virtuoso) 主要用于语义网和知识图谱, 侧重于数据的互操作性和标准化。

主要代表:

- Apache Jena: 一个 Java 框架, 支持 RDF 数据模型和 SPARQL 查询语言。
- Virtuoso: 一个多功能数据库, 支持 RDF 和 SPARQL, 常用于语义网应用。
- Stardog: 一个商业图数据库, 支持 RDF、SPARQL 和 OWL。

特点:

- 模型: RDF (资源描述框架) 使用三元组 (主语-谓词-宾语) 来表示数据。数据通常是语义网的数据。
- 查询语言: SPARQL, 用于查询 RDF 数据。
- 用途: 适合于处理语义网数据、知识图谱和互联数据。

3.基于 OWL 本体语言的图数据库

OWL (如 Protégé) 则用于定义和管理复杂的本体, 提供更高级的语义描述和推理能力。

主要代表:

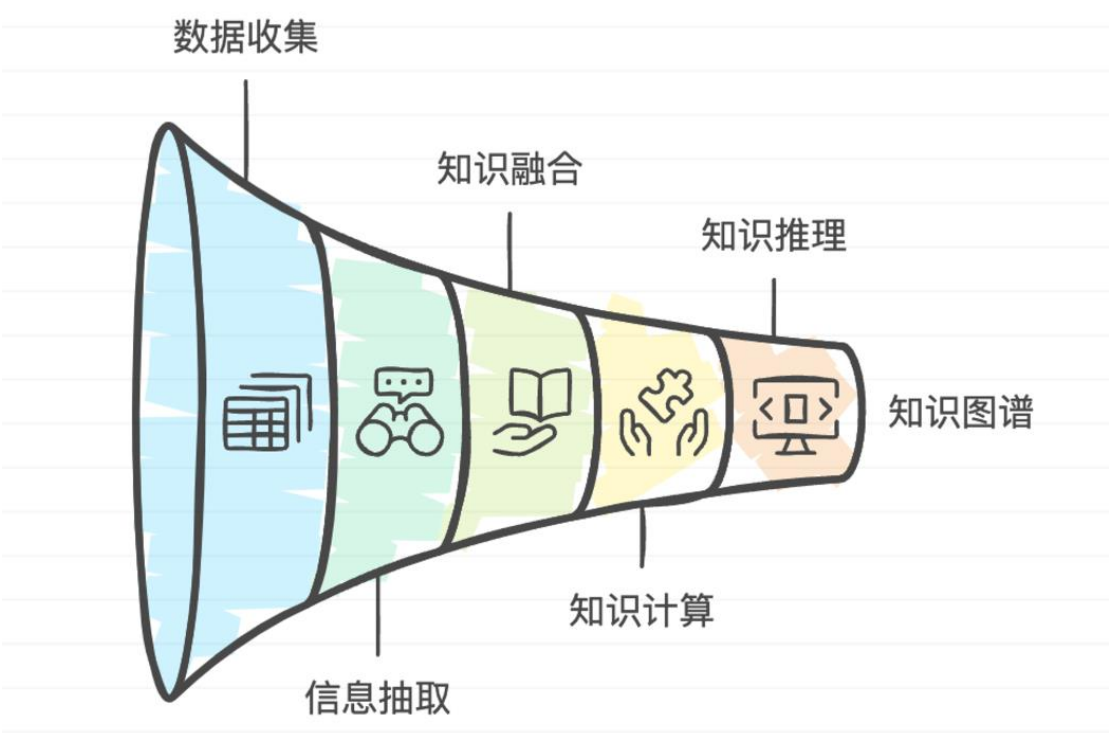
- Protégé: 一个用于构建 OWL 本体的开源工具, 允许创建和管理本体。
- OntoText GraphDB: 一个支持 OWL 和 RDF 的图数据库, 提供高效的查询和推理能力。

特点:

- 模型：OWL（Web 本体语言）用于定义和处理本体，包括类、属性和实例的复杂关系。OWL 是 RDF 的扩展，提供更强的表达能力。
 - 查询语言：通常与 RDF 数据结合使用，查询语言为 SPARQL。
 - 用途：适合于需要复杂推理和语义处理的应用，如知识图谱和领域本体。

4. 知识图谱的知识抽取

知识抽取（information extraction）是构建知识图谱的第一步，为了从异构数据源中获取候选知识单元，知识抽取技术将自动从半结构化和无结构数据中抽取实体、关系以及实体属性等结构化信息。



4.1. 知识图谱的构建

- 实体抽取（entity extraction）:是指从文本中识别出具有特定意义的实体，并将其转换为知识图谱中的节点。
实体抽取是知识图谱构建的基础，实体抽取的准确性直接影响到知识图谱的构建质量。实体抽取的主要任务包括识别实体类型、实体边界和实体关系。
- 关系抽取（relation extraction）：是指从文本中识别出实体之间的关系，并将其转换为知识图谱中的边。关系抽取是知识图谱构建的关键，关系抽取的准确性直接影响到知识图谱的构建质量。关系抽取的主要任务包括识别关系类型、关系边界和关系方向。
- 属性抽取（attribute extraction）:是指从文本中识别出实体的属性，并将其转换为知识图谱中的属性。属性抽

取是知识图谱构建的重要组成部分，属性抽取的准确性直接影响到知识图谱的构建质量。属性抽取的主要任务包括识别属性类型、属性边界和属性值。

- 知识融合 (knowledge fusion) :是指将多个知识图谱中的知识进行合并，以形成一个统一的知识图谱 知识融合是知识图谱构建的重要组成部分，知识融合的准确性直接影响到知识图谱的构建质量。知识融合的主要任务包括识别实体类型、实体边界和实体关系。
- 知识加工 (knowledge processing) :是指对知识图谱中的知识进行处理，以形成一个统一的知识图谱。知识加工是知识图谱构建的重要组成部分，知识加工的准确性直接影响到知识图谱的构建质量。知识加工的主要任务包括识别实体类型、实体边界和实体关系。
- 知识推理 (knowledge reasoning) :是指对知识图谱中的知识进行推理，以形成一个统一的知识图谱。知识推理是知识图谱构建的重要组成部分，知识推理的准确性直接影响到知识图谱的构建质量。知识推理的主要任务包括识别实体类型、实体边界和实体关系。

4.2. 结合大语言模型的知识抽取

在没有使用 LLM 的情况下，知识抽取主要依赖于人工规则和启发式方法，这些方法通常需要领域专家手动编写规则，并且难以处理复杂的关系和语义。而且针对不同的步骤需要不同的算法，给知识抽取带来了很大的难度。而大语言模型（LLM）的出现，给知识抽取带来了新的思路，它本身具有通用知识，然后通过少量的 few shot，就可以在特定的领域进行应用。这个也是 graphRAG 的最大作用，它就是实现了这么一套流程，在文本中实现知识图谱的抽取等操作。

4.3. LLM 抽取存在问题和探讨方向

质量评估也是知识库构建技术的重要组成部分，这一部分存在的意义在于：可以对知识的可信度进行量化，通过舍弃置信度较低的知识来保障知识库的质量。整体的 entity，relation，attribute 的准确性，多样性，全面性。通过统计和图形可视化等量化或直观化评估等。评估指标如下：

4.3.1 实体识别的准确性

a.每个 chunk 中 token 的数量，以及 token 个数的分布

```
```cypher
```

```
MATCH (n:__Chunk__) RETURN n.n_tokens as token_count, count(*) AS count
```

```
...
```

b.每个 chunk 中抽取的 entity 数量

```
```cypher
```

```
MATCH (d:__Chunk__)
```

```
RETURN count {(d)-[:HAS_ENTITY]->()} AS entity_count
```

```
...
```

c.抽样检查实体描述（看是否乱码，是否相对正确）

```
```cypher
```

```
MATCH (n:__Entity__) RETURN n.name AS name, n.description AS
```

```
description LIMIT 1
```

```
...
```

d. 每个实体的关联度（判断最热的那些实体，是否满足预期）

```
```cypher
```

```
MATCH (n:__Entity__)
```

```
RETURN n.name AS name, count{(n)-[:RELATED]-()} AS degree
```

```
ORDER BY degree DESC LIMIT 5
```

```
...
```

e. 实体属性的正确性（是否必须存在的属性在，是否有错误属性等）

4.3.2 关系抽取的准确性

a.relationship 种类数量

```
```cypher
```

```
...
```

b.relationship 描述（看是否乱码，是否相对正确）

```
```cypher
    MATCH ()-[n:RELATED]->() RETURN n.description AS description LIMIT
5
    ```
```

c. relationship 种类的分布

d. 关系属性的正确性（是否必须存在的属性在，是否有错误属性等）

#### 4.3.3 communtiy detection 的准确性

a. 社区的各个属性信息（title, summary, full\_content）（看是否乱码，是否相对正确）

b. 社区的划分和社区的规模

c. 社区的等级和分布

cumminity 等级分布情况

```
MATCH (n: __Community__)
```

```
RETURN n.level AS level, count{(n)-[:!IN_COMMUNITY]-{}} AS members
```

#### 4.4. 知识图谱的知识更新

从逻辑上看，知识库的更新包括概念层的更新和数据层的更新。

概念层的更新是指新增数据后获得了新的概念，需要自动将新的概念添加到知识库的概念层中。数据层的更新主要是新增或更新实体、关系、属性值，对数据层进行更新需要考虑数据源的可靠性、数据的一致性（是否存在矛盾或冗杂等问题）等可靠数据源，并选择在各数据源中出现频率高的事实和属性加入知识库。知识图谱的内容更新有两种方式：

- 全面更新：指以更新后的全部数据为输入，从零开始构建知识图谱。这种方法比较简单，但资源消耗大，而且需要耗费大量人力资源进行系统维护；
- 增量更新：以当前新增数据为输入，向现有知识图谱中添加新增知识。这种方式资源消耗小，但目前仍需要大量人工干预（定义规则等），因此实施起来十分困难。

**graphRAG 在 0.4.0 版本中也引入增量更新功能, 让与图书库等的集成变得更加方便。**

## **第二章：GraphRAG 整体概述**

有了知识图谱的基本知识之后，我们来快速了解一下 Graphrag，看它是如何通过知识图谱这个技术来帮助在大语言应用最广泛的一个场景—RAG 的。两者的有机结合就是 Graphrag。

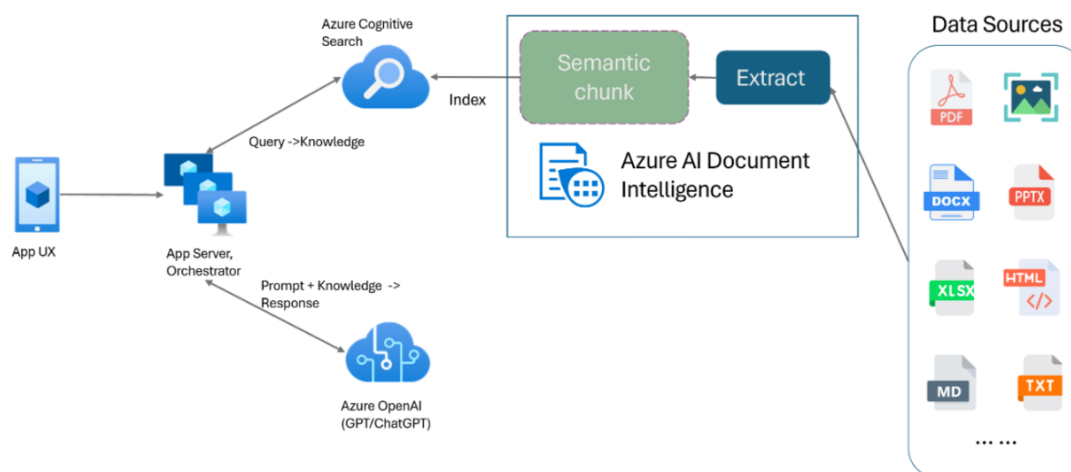
GraphRAG（图基检索增强生成），是由微软研究院开发的一种新型检索增强生成（Retrieval Augmented Generation）框架。它旨在利用知识图谱和大型语言模型（LLMs），来提升信息处理和问答能力。GraphRAG 通过



构建知识图谱，从非结构化文本中提取结构化数据，让模型能够更好地理解和处理复杂信息。

## 1. GraphRAG 主要解决什么问题？

Baseline RAG（标准 RAG）是一种基础版本的检索增强生成架构，采用前置处理流程，以文档为中心，没有做其他的内容洞见，通过 chunk 方式来切割。



而 GraphRAG 是一种结构化的、分层的检索增强生成（RAG）方法，与使用纯文本片段的朴素语义搜索方法相反，GraphRAG 过程涉及从原始文本中提取知识图谱、构建社区层次结构、为这些社区生成摘要，然后在执行基于 RAG 的任务时利用这些结构。

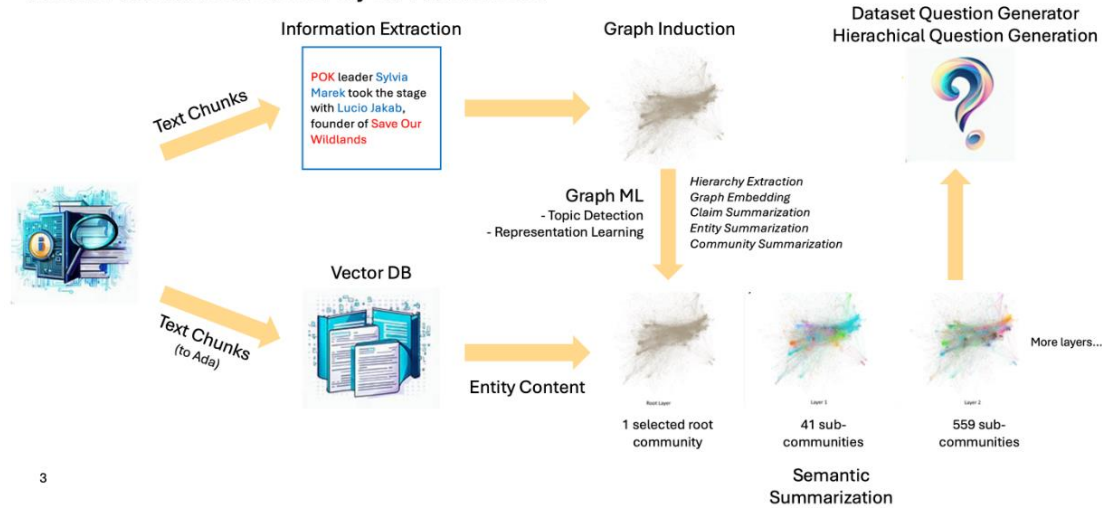
当面临以下情况时，GraphRAG 比 baseline RAG 有更好的表现：

- 当一些实体（名词）占比比较小，通过 baseline RAG 无法正确召回时；
- 当希望通过实体关系正向和反向查询内容，通过实体关系实现很正确召回时；
- 通过 Global Search 对整体进行总结和洞见时。

## 2. GraphRAG 的数据处理流程

GraphRAG 流程的基本步骤如下：

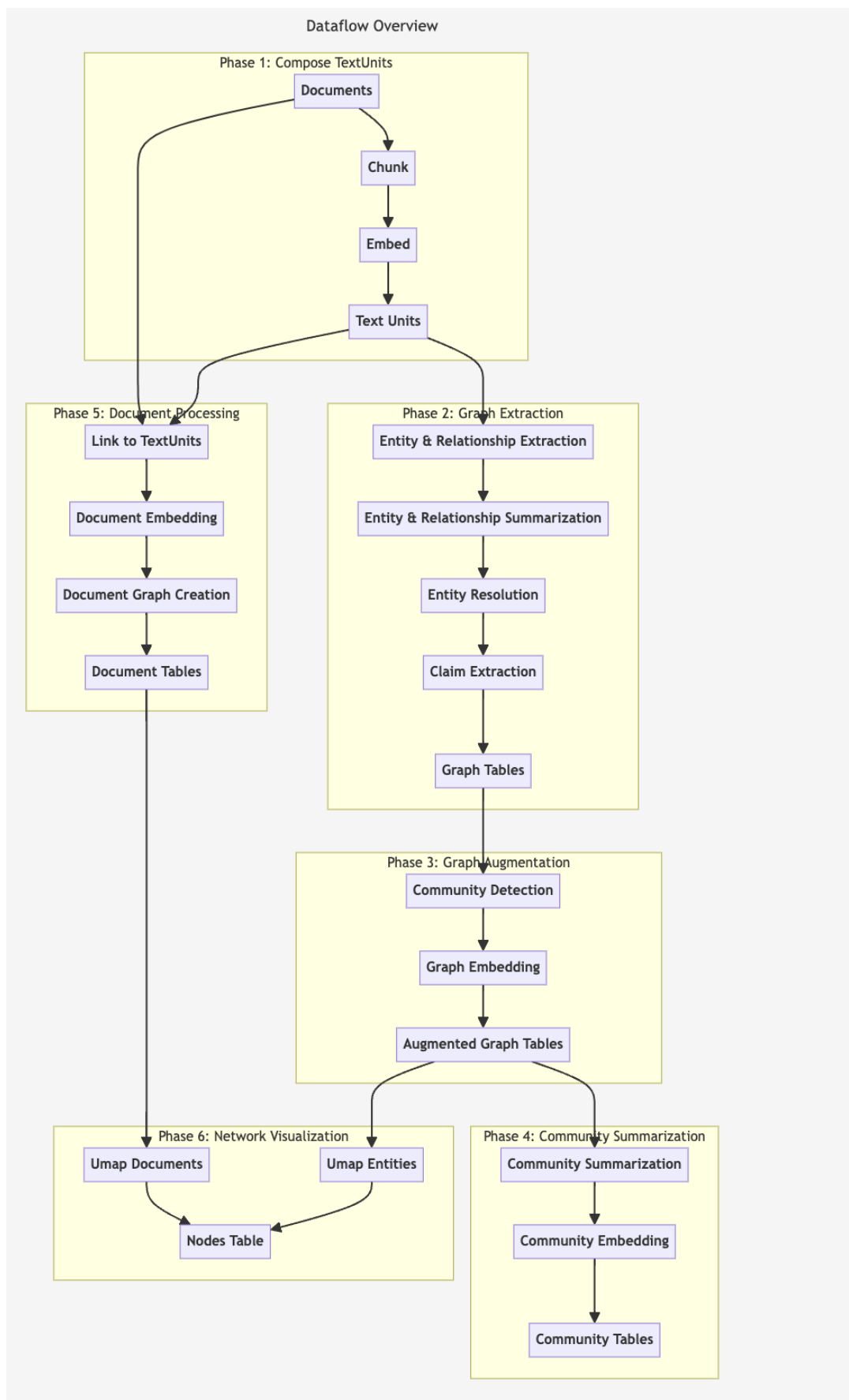
## Indexed Information Discovery on Private Data



3

1. Slice up an input corpus into a series of TextUnits, which act as analyzable units for the rest of the process, and provide fine-grained references into our outputs.  
将输入语料库切成一系列 TextUnits，这些 TextUnits 充当流程其余部分的可分析单元，并在我们的输出中提供细粒度的引用。
2. Extract all entities, relationships, and key claims from the TextUnits using an LLM.  
使用 LLM 来抽取这些实体，关系，主要申明
3. Perform a hierarchical clustering of the graph using the Leiden technique. To see this visually, check out Figure 1 above. Each circle is an entity (e.g., a person, place, or organization), with the size representing the degree of the entity, and the color representing its community.  
使用 Leiden 技术对图形执行分层聚类。要直观地看到这一点，请查看上面的图 1。每个圆圈都是一个实体（例如，一个人、一个地方或组织），大小代表实体的程度，颜色代表其社区。
4. Generate summaries of each community and its constituents from the bottom-up. This aids in holistic understanding of the dataset.  
自下而上生成每个社区及其成员的摘要。这有助于全面了解数据集。

更加详细的步骤参照：



### 3. GraphRAG 的查询接口

GraphRAG 的查询接口主要有 Local Search 和 Global Search。

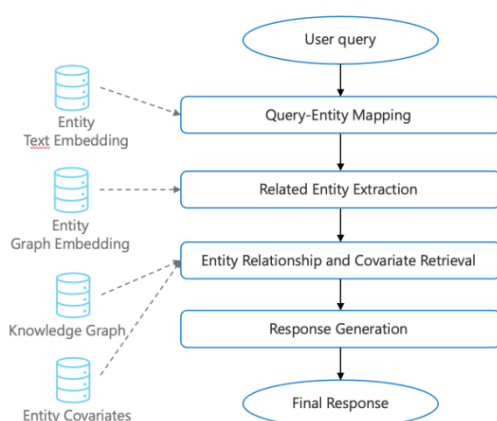
在 0.4.0 版本中新增加了 Drift search。

### 3.1. Local Search 的实现流程

本地搜索方法，将知识图谱中的结构化数据与输入文档中的非结构化数据相结合，能够在查询时使用相关实体信息来增强 LLM 上下文。它非常适合回答需要了解输入文档中提到的特定实体的问题。本地搜索方法从知识图谱中识别一组在语义上与用户输入相关的实体。用这些实体充当知识图谱的访问点，可以提取更多相关详细信息，例如连接的实体、关系、实体协变量和社区报告。

此外，它还能从已识别实体关联的原始输入文档中提取相关文本块，对这些候选数据源进行优先级排序和筛选，以适合预定义大小的单个上下文窗口。

This method is best suited for answering questions related to **specific entities** that have a small number of related entities.



#### Step 1: Query-entity Mapping

Extract top-N graph entities that have highest **semantic relevance** to the user query by computing the similarity score between the **text embeddings** of the user query and the entity description.

#### Step 2: Related Entity Extraction

For each target entity E extracted from step 1, find the top-K neighbors with the highest **behavioral relevance**. This can be computed as the similarity score of the **graph embeddings** of E and other entities in the graph, or as the degree of the direct connections of E.

#### Step 3: Entity Relationship and Covariate Retrieval

Given the entity set extracted from steps 1+2, retrieve all covariates associated with these entities (e.g., claims), and all records of relationships between these entities. Use this data to construct the context for answering the user query.

#### Step 4: Response Generation

Given the user query and the data context constructed in step 3, use a generative AI model to generate a response.

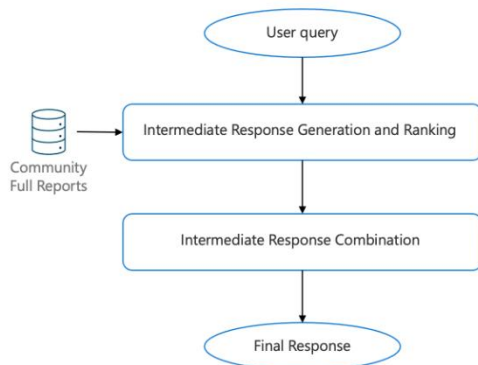
### 3.2. Global Search 的实现流程

baseline RAG 致力于对数据集中语义相似的文本内容的向量搜索，难以处理需要聚合数据、集中信息以组成答案的查询。诸如“数据中排名前 5 位的主题是什么？”

但使用 GraphRAG，我们就可以回答这些问题，因为生成的知识图谱结构 LLM 能够告诉我们，整个数据集的结构（以及主题）。这需要将私有数据集组织成预先汇总的、有意义的语义集群，使用全局搜索方法来进行查询。

全局搜索方法，使用来自图形社区层次结构的、指定级别的、生成社区报告的 LLM 集合作为上下文数据，以 map-reduce 方式生成响应。在该 map 步骤中，社区报告被分割为预定义大小的文本块，每个文本块用于生成包含点列表的中间响应，每个点列表都伴随着一个数字评级，指示该点的重要性。在该 reduce 步骤中，从中间响应中筛选出的一组最重要的点，被聚合并用作生成最终响应的上下文。

This method is best suited for answering questions that require [summarization of the dataset as a whole](#).



#### Step 1: Intermediate Response Generation and Ranking

Shuffle and partition all community reports into  $N$  non-overlapping chunks, such that each chunk can be accommodated within a fixed-size context window.

For each of the  $N$  chunks, use a generative AI model to generate a response to the user question, along with a numerical score that indicates the quality of the answer. Rank the answers by the quality score and discard any answer with score below a predefined threshold.

#### Step 2: Intermediate Response Combination

Combine the ranked intermediate responses from step 1 into a single context window and use a generative AI model to produce the final response.

### 3.3 Drift Search 介绍

Drift Search 具有灵活遍历的动态推理，结合了全局搜索和本地搜索的特征，以平衡计算成本和质量结果的方法生成详细的响应。因为我们知道在全局搜索过程需要遍历某一层的社区，通过 map reduce 来对内容进行全面总结回答，如果对应的知识图谱巨大的，社区很多的话，需要消耗大量的 token，花费很长时间才能得到最终答案。但是 local search 和 baseline rag 一样，只查找相关实体缩关联的社区信息，构建查询上下文，对于全局类型的问题不能很好的回答。Drift Search 就是在两种采取一个权衡。

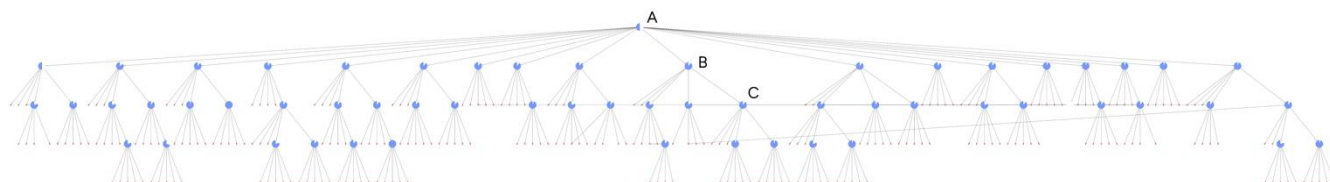
DRIFT Search 通过在搜索过程中包含社区信息（不是实体）。这极大地扩展了查询起点的广度，最终答案中检索和使用更多种类的事实。此新增功能通过为本地搜索提供更全面的选项来扩展 GraphRAG 查询引擎，同时该选项使用社区见解将查询细化为详细的后续问题。

DRIFT Search 首先当用户提交查询时，DRIFT 会将其与语义上最相关的前  $K$  个社区报告进行比较。这一过程生成了初步答案以及若干后续问题，后续问题类似于简化版的全局搜索。为实现这一点，我们使用假设文档嵌入 (Hypothetical Document Embeddings, 简称 HyDE) 扩展查询，以提高敏感度（召回率）。具体步骤包括：对查询进行嵌入，将查询与所有社区报告进行匹配，选出前  $K$  个最相关的结果，然后利用这些结果尝试回答查询。这一方法的目标是利用高层次抽象来指导进一步的探索。

接着在初步处理完成后，DRIFT 使用本地搜索变体执行每个后续查询。这将生成额外的中间答案和新的后续问题，形成一个不断优化的循环，直到搜索引擎满足其终止条件。目前，终止条件设置为两轮迭代（未来研究将探索通过奖励函数引导终止的可能性）。这一阶段体现了基于全局信息的查询优化。通过使用全局数据结构，即使初始查

询偏离了索引的设定角色，DRIFT 仍能够在知识图谱中导航至具体且相关的信息。这种后续查询的过程使 DRIFT 能够根据新发现的信息调整其方法。

最后输出是一个按与原始查询相关性排序的问题和答案层级结构。该层级结构可以根据特定用户需求进行自定义。在基准测试中，使用了一种简单的 Map-Reduce 方法来汇总所有中间答案，每个答案的权重相同。



**DRIFT Search** 动态结合全局洞察与局部优化，使其能够从高层次摘要导航到知识图谱中的原始文本块。这种分层方法确保即使初始查询偏离了索引时使用的角色设定，也能保留详细且具有上下文的信息。

通过将广泛的问题分解为细化的后续查询，DRIFT 捕获了精细的细节，并根据不断变化的上下文进行调整，从而适应多种查询类型。这种能力使其在处理既需要广度又需要深度的查询时表现尤为出色，同时不会丢失特定的细节信息。

DRIFT 的未来版本将包含全局搜索的改进，使其能够更直接地解决当前由全局搜索提供最佳服务的问题。希望随后转向一个可以处理本地和全球问题的单一查询接口。将进一步发展 DRIFT 的终止逻辑，可能通过平衡新信息与冗余的奖励模型。此外，使用全局或本地搜索模式执行后续查询可以提高效率。某些查询需要更广泛的数据访问，这可以通过利用查询路由器和轻量级**全局搜索**变体来实现，该变体使用较少的社区报告、令牌和整体资源。



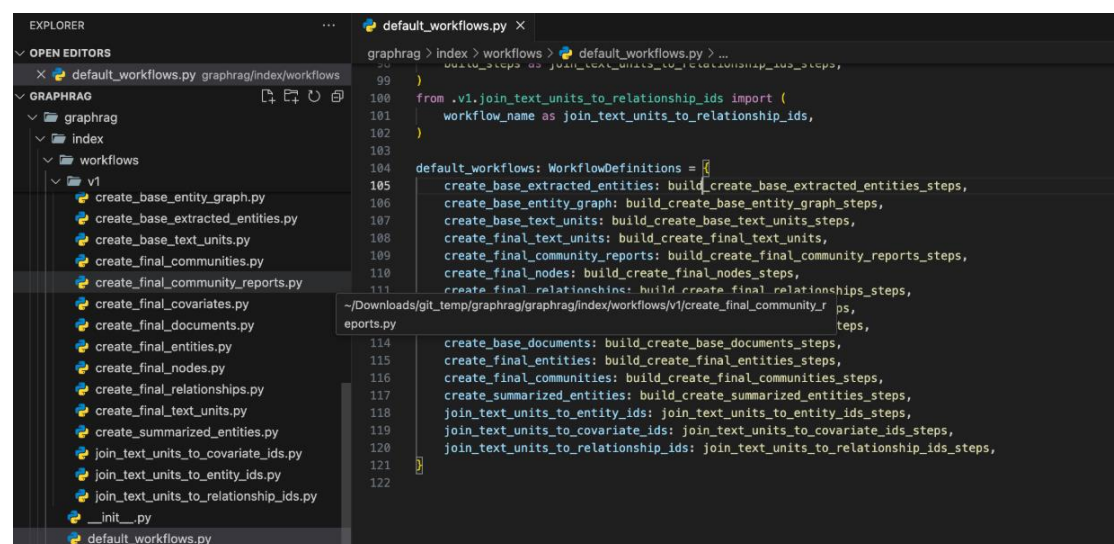
## 第三章：GraphRAG 实践过程中面临的挑战

在 GraphRAG 使用过程中，用户反馈有价格贵、速度慢、如何评估、如何实现增量、技术如何混合等诸多挑战。

### 1. 价格贵

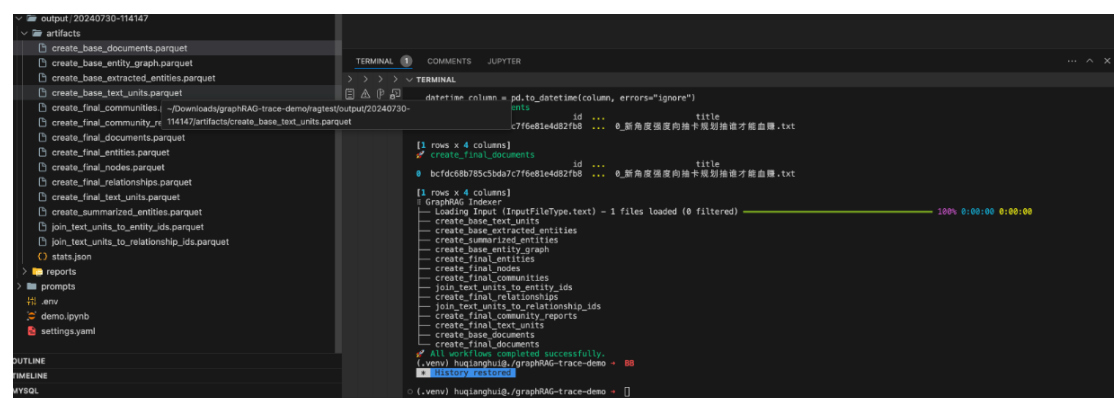
初接触 GraphRAG，往往会觉着 GraphRAG 很贵。因为当时大概率主要使用 gpt-4-turbo 或者 gpt-4o 模型。后面随着 gpt-4o-mini 的 API 推出，成本大大下降。但在实践使用过程中，还需要提前计算 token 的数量级，来预判所需要的成本。要计算前期的索引成本，首先要弄清楚索引过程中做了哪些事情，产生了哪些内容，以及这些内容的作用是什么。

#### 1.1. 首先通过以下文件了解 workflow 定义。



```
graphrag > index > workflows > default_workflows.py > ...
99
100
101 from .v1.join_text_units_to_relationship_ids import (
102 workflow_name as join_text_units_to_relationship_ids,
103)
104
105 default_workflows: WorkflowDefinitions = {
106 create_base_extracted_entities: build_create_base_extracted_entities_steps,
107 create_base_entity_graph: build_create_base_entity_graph_steps,
108 create_base_text_units: build_create_base_text_units_steps,
109 create_final_text_units: build_create_final_text_units,
110 create_final_community_reports: build_create_final_community_reports_steps,
111 create_final_nodes: build_create_final_nodes_steps,
112 create_final_relationships: build_create_final_relationships_steps,
113 create_base_documents: build_create_base_documents_steps,
114 create_final_entities: build_create_final_entities_steps,
115 create_final_communities: build_create_final_communities_steps,
116 create_summarized_entities: build_create_summarized_entities_steps,
117 join_text_units_to_entity_ids: join_text_units_to_entity_ids_steps,
118 join_text_units_to_covariate_ids: join_text_units_to_covariate_ids_steps,
119 join_text_units_to_relationship_ids: join_text_units_to_relationship_ids_steps,
120 }
121
122
```

#### 1.2. 然后通过运行来构建索引。



```
date(iso column = pd.to_datetime(column, errors="ignore")
...
id ... title
c7f6e81e4d82f8b ... 0_新角度强度向陆卡规划措施才能血牌.txt

[1 rows x 4 columns]
create_final_documents
id ... title
bcfd68b785c5da7c7f6e81e4d82f8b ... 0_新角度强度向陆卡规划措施才能血牌.txt

[1 rows x 4 columns]
GraphRAG Indexer
Loading Input (InputFileType.txt) ~ 1 files loaded (0 filtered) 100% 0:00:00 0:00:00
create_base_extracted_entities
create_summarized_entities
create_base_entity_graph
create_final_entities
create_final_nodes
create_final_communities
join_text_units_to_entity_ids
create_final_relationships
join_text_units_to_relationship_ids
create_final_community_reports
create_final_text_units
create_base_documents
create_final_documents
All workflow completed successfully.
[env] huqianghui@graphrag-trace-demo
```

在运行过程中，如果选择 prompt tune，它会产生适应于文本的 prompt，如果没有就使用

默认的 prompt，来多次请求 LLM 而消费 token，查看 ragtest/prompts 里面 prompt 文件。文件列表如下：

```
entity_extraction.txt
summarize_descriptions.txt
claim_extraction.txt
community_report.txt
```

据文件名字和列表推测，token 的消费和文本里面 entity，community 和 claim 数量相关。通过监控调用次数，如果不修改 prompt，token 消费公式大致如下：

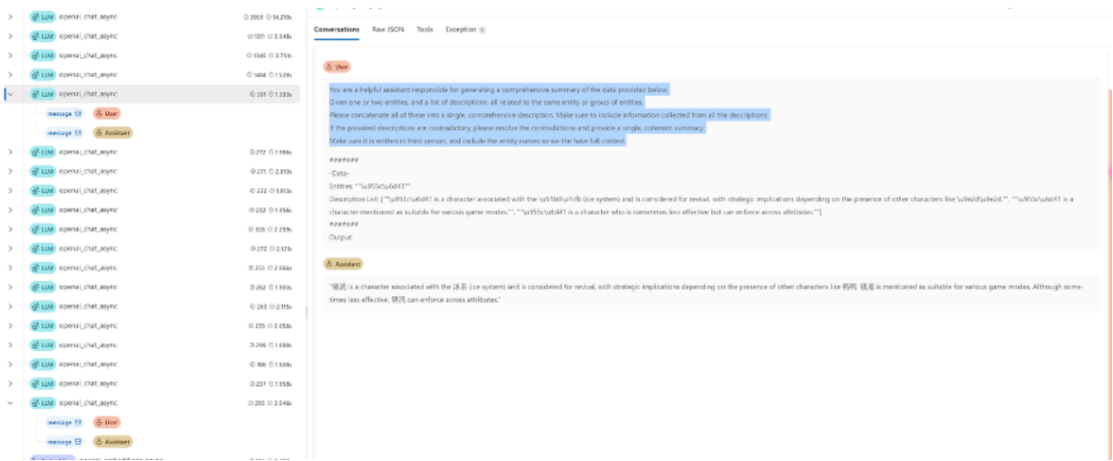
Token Consumption formula:

```
Base Call Count= Tokens of Document / chunk size

Entity Extraction Token Consumption = (2200 + chunk) * (Base Call Count)
Entity Description Token Consumption = (130 + Entity Description) * (Entity Count)
Community Token Consumption = (1900 + Community Description) * (Community Count)
Claim Extraction Consumption = (1200 + chunk) * (Base Call Count) optional ; 默认不抽取，需要打开配置项才抽取
```

前面的数字，是默认的 prompt template 通过 gpt-turbo c100K 对应的编码器的 token 数字，如果你修改了模型，或者自定义 prompt 都需要相应的变化。

下图是引入 prompt-flow SDK 展示出来的调用链路图：



上面的公式就是根据这个链路的数据推导出来的。

所以在使用之前，大家可以根据这个公式来估算一下总要消耗的 token 数量，也可以用一个小文本的内容，进行一个简单测试。



2. 实际使用速度慢

在实际使用过程中，GraphRAG 可能会显得较慢，从代码层面来看主要有主要有如下几个步骤：

- 2.1. 初始化时间：加载知识图谱。
- 2.2. 查询复杂度：通过向量查询实体，构建 context。
- 2.3. LLM 调用：调用 LLM，生成 token，有流模式和非流模式。

下图是一个运行时调用的完整链路：

Collection: local\_search\_2024-08-01134456

Traces

Q: Input filter condition in python syntax. e.g. name == "web\_classification" and cumulative\_token\_count.total > 1000

Auto Refresh: OFF 10s 1m 5m Dark Theme

Kind	Name	Input	Output	Start time	End time	Latency	Total tokens	Status
Function	get_local_search_engine	reports: [ ... ] text_units: [ ... ] entities: [ ... ] relationships: [ ... ] covariates: [ ... ]	<graphrag.query.structured_search.local_search.local_search object at 0x30...	03/08/2024, 01:44:57 pm	03/08/2024, 01:44:57 pm	0.001s		●
Function	read_indexer_relationships	{ final_relationships: " source target weigh }	{ 7: { ... } 8: { ... } 9: { ... } 10: { ... } }	03/08/2024, 01:44:56 pm	03/08/2024, 01:44:56 pm	0.006s		●
Function	read_indexer_text_units	{ final_text_units: " id ... covariate_id }	{ 0: { ... } 1: { ... } }	03/08/2024, 01:44:56 pm	03/08/2024, 01:44:56 pm	0.003s		●
Function	read_indexer_reports	{ final_community_reports: " community full final_nodes: " level title type ... top_in community_level: 2 }	{ 0: { ... } 1: { ... } }	03/08/2024, 01:44:56 pm	03/08/2024, 01:44:56 pm	0.007s		●
Function	read_indexer_covariates	{ final_covariates: " id human_readable_id c }	{ 0: { ... } 1: { ... } 2: { ... } 3: { ... } 4: { ... } }	03/08/2024, 01:44:56 pm	03/08/2024, 01:44:56 pm	0.004s		●
Function	read_indexer_entities	{ final_nodes: " level title type ... top_in final_entities: " id ... description_embed community_level: 2 }	{ 0: { ... } 1: { ... } 2: { ... } 3: { ... } 4: { ... } 5: { ... } }	03/08/2024, 01:44:56 pm	03/08/2024, 01:44:56 pm	0.407s		●
Function	_get_embedding_description_store	{ vector_store_type: "lancedb" config_args: { ... } }	<graphrag.query.structured_search.lancedb.LanceDBVectorStore object at 0x202294c10>	03/08/2024, 01:44:56 pm	03/08/2024, 01:44:56 pm	0.000s		●
Function	_configure_paths_and_settings	{ data_dir: null root_dir: ".../ragtest" }	C:\Users\hpq\Downloads\graphdemo\graphdemo\output\20240802-172606\entit...	03/08/2024, 01:44:56 pm	03/08/2024, 01:44:56 pm	0.020s		●
Function	LocalSearch.search	{ query: "Who is Scrooge, and what are his m }	(Response): " Scrooge and His Main Relationships# Who is Scrooge?# [ ... ]	03/08/2024, 01:43:13 pm	03/08/2024, 01:43:13 pm	15.435s	11	●
Function	LocalSearch.search	{ query: "Who is Scrooge, and what are his m }	(Response): "; context_data: (Reports): id tit...	03/08/2024, 01:44:18 pm	03/08/2024, 01:44:20 pm	1.760s	11	●
Function	LocalSearch.search	{ query: "Who is Scrooge, and what are his m }	(Response): "; context_data: (Reports): id tit...	03/08/2024, 01:43:19 pm	03/08/2024, 01:43:21 pm	1.840s	11	●

从时间花费来看，主要还是在调用大语言模型本身。因为 graphrag 的返回，某人是段落式，回答的内容比较多，所以如果想优化这个时间的话，可以使用自己的 prompt，修改返回模式，让对一些问题回答更简单一些。这也是另外一个需要在实际使用过程的一个权衡点。

3. 难以评估 GraphRAG 的好坏

GraphRAG 和 baseline RAG 对知识的处理不一样，GraphRAG 是对知识进行二次加工，适合知识点跳动，需要串联或者全局查找。所以，除了基本评价指标外，还需要一个更加贴近技术的评价指标，来评价最终结果的好坏，这些指标还需要进一步探索。

目前，主要参数的指标方向有如下几个：

### 3.1. 信息整合

将文档中不同部分的相关概念整合在一起。

### 3.2. 关系分析

增加了文档中未明确说明的概念之间的关系和互动。

### 3.3. 结构重组

将原有的章节结构重新组织成更连贯的分析文本。

### 3.4. 信息扩展

基于文档中的概念增加一些推断信息或解释。

由于 GraphRAG 的效果，主要取决于知识图谱本身的质量，也需要构建一个评级体系测量和验证这些点 (node)，边 (edge)，社区 (community) 和 协变 (Covariate, Claim) 的质量。

目前考虑到的主要节点的数量，以及节点边的数量。其实就是热点节点 topK 中，是否和业务认识的一直。其次就是社区的数据，社区的分布等等。

也需要结合具体的业务场景，和业务人员一起来构建一个更合适的标准。

如果出现了偏差，目前能优化的点主要就是： 1) 提示词优化 2) 增加大语言模型的理解次数。

### 第三章：GraphRAG 的图形化展示

在第二章提出了图谱的质量，其中最直观的方式还是有一个比较友好图形来展示结果。

它可以很直观的看到：点，边，社区等信息。其次，就是提供给 Local Search 中对应的 entityID, Global Search 中的 community reportID 引用信息，并根据这些 ID 获取进一步信息，进行查询和可视化展示。

如下, 在一个 Local Search 中引用 317 实体, 在对应 entity 表中就能通过 human readable id 查询对应的实体。

```
Winston is a complex character from George Orwell's novel "Nineteen Eighty-Four." He is a member of the Outer Party and works at the Ministry of Truth, where his job involves altering historical records to fit the Party's propaganda. Winston is deeply resentful of the Party and Big Brother, harboring a sense of intellectual inferiority and mental domination by figures like O'Brien. His life is marked by intense fear, paranoia, and physical and emotional suffering, particularly during his interactions with O'Brien, who tortures and interrogates him [Data: Entities (8, 3)].

Winston's reflections on reality, power, and the nature of suffering are central to his character. He contemplates the symbolic meaning of chess, the principles of Ingsoc, and the dominion of The Party. His dreams and memories, including the tragic death of his mother and sister, the bells of St. Clement's, and his early childhood, provide a poignant backdrop to his internal struggles [Data: Entities (8)].

Interactions and Relationships

Julia
Winston's most significant relationship is with Julia, a fellow Party member. Despite the risks, they engage in a love affair, renting a room from Mr. Charrington for their secret meetings. Julia represents a source of intimacy and rebellion for Winston, as they navigate their dangerous liaison under the watchful eyes of The Party [Data: Entities (8)].

O'Brien
O'Brien is another crucial figure in Winston's life. Initially, Winston feels a sense of connection and admiration towards O'Brien, hoping that his political orthodoxy is not perfect. However, O'Brien ultimately becomes his torturer, subjecting Winston to severe psychological and physical pain. Despite this, Winston experiences moments of connection and even a peculiar intimacy with O'Brien [Data: Entities (8)].

Mr. Charrington
Mr. Charrington is the shop owner who rents a room to Winston and Julia for their secret meetings. Initially, he appears discreet and non-judgmental, but later reveals a more authoritative and alert persona, indicating his role in the Party's surveillance [Data: Entities (317)].

Other Characters
Winston also interacts with various other characters, such as Syme, Parsons, and the old man in the pub. These interactions reveal his curiosity about the past and the changes brought about by The Party. For instance, Syme is a colleague who discusses the principles of Newspeak with Winston, while Parsons is a fellow employee at the Ministry of Truth [Data: Entities (8, 83)].

Conclusion
Winston is a deeply reflective and observant character, constantly grappling with the oppressive nature of The Party and his own internal conflicts. His interactions with Julia, O'Brien, Mr. Charrington, and others provide a multifaceted view of his struggles and the dystopian world he inhabits. Through these relationships, Winston's character is fleshed out, revealing the complexities of life under totalitarian rule.
```

human\_readable\_id:

Gephi 0.10.1 - Untitled 1

Overview

Data Laboratory

Preview

clustered\_graph x

Data Table x

Nodes Edges

Configuration

Add node

Add edge

Search/Replace

Import Spreadsheet

Export table

More actions

id	Label	Interval	level	cluster	id	hum
"ST CLEMENT'S"	"ST CLEMENT'S"	1	43	4b8aa587c7344adac2cbfa69d5e4...	313	
"ST MARTIN'S"	"ST MARTIN'S"	1	43	52701d941d1b45359693baae8f2670...	314	
"VICTORY SQUARE"	"VICTORY SQUARE"	1	43	31499ee6277a4d71b19cb5b6be554...	315	
"THE PICTURE GALLERY"	"THE PICTURE GALLERY"	1	43	d99eab5d5fd47278692569d2a939...	316	
"MR CHARRINGTON"	"MR CHARRINGTON"	1	41	d53f15cb7f7845de91cc4ad44f9f6a...	317	
"ST MARTIN'S-IN-THE-FIELDS"	"ST MARTIN'S-IN-THE-FIELDS"	1	43	23becf8c6fca4f47a53ec4883d4b63f...	318	
"THE CENTRE"	"THE CENTRE"	1	41	d0f1a3bdc12342589539f4956d19f561...	319	
"THE SHOP"	"THE SHOP"	1	41	ac41b77ba33a4c84877ab42f5aba03...	320	
"THE FLAT"	"THE FLAT"	1	100	5d3184dadfd647a5a7e565f72c60f04...	321	
"THE WOMAN ON THE TELESCRE..."	"THE WOMAN ON THE TELESCRE..."	1	100	0ec262c2cfe4dd5813655e5e59631...	322	

默认的图形数据主要记录在 .parquet 文件中。

## 读取 parque 文件，通过图形库展示

### 1. 在 notebook 中，使用 yFiles Graphs for Jupyter 可视化内容

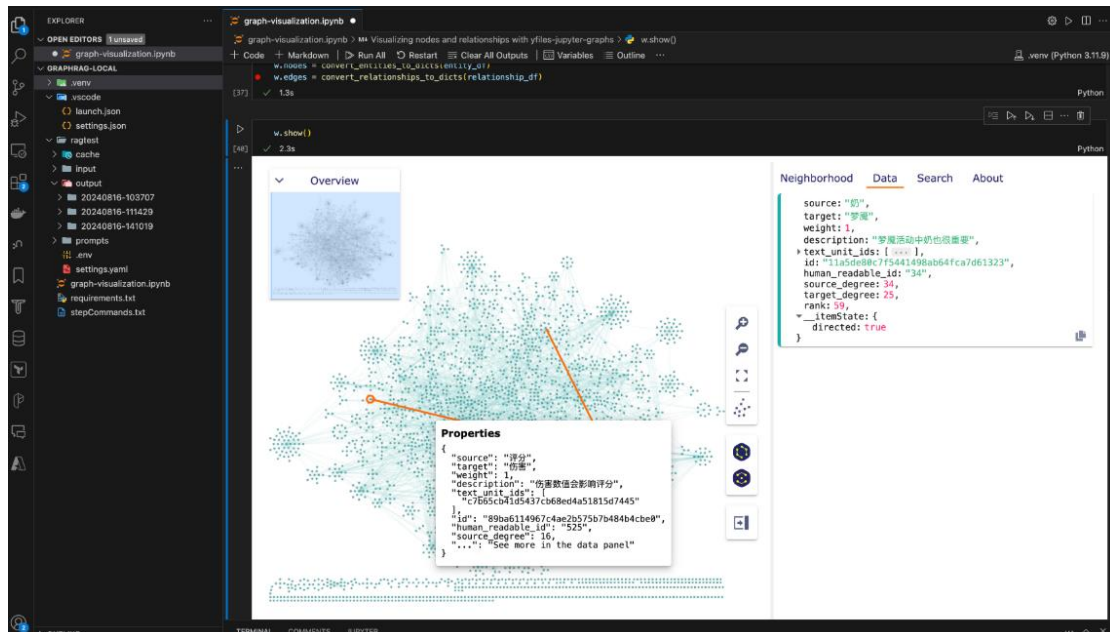
最简单的方式就是使用 yFiles\_jupyter\_graphs。它是一个用于在 Jupyter Notebook 中可视化和操作图形结构的 Python 包，能够强大的图形可视化和编辑功能，适用于需要处理图形结构的场景。通过 GraphWidget，可以在 Notebook 中交互式展示和操作图形，添加节点、边，或应用布局算法。以下是简单的使用步骤。

交互功能：

- 通过鼠标操作节点和边，可以直接在图形上进行交互，比如拖动节点、点击边等。
- 支持图的缩放、平移等功能，便于查看复杂结构。

适用场景：

- 网络结构展示：适合展示如社交网络、依赖图等复杂结构。
- 流程图和拓扑图：可以用于绘制流程图和各类拓扑图。
- 算法研究：可用于图论算法的可视化展示和调试。



## 2. 在 web 中，使用 plotly 可视化内容

### 2.1 在 Python 中使用 plotly

在 Python 中使用 `plotly.subplots.make_subplots` 和 `go.Figure` 的 `show()` 函数时，可通过以下几种方式显示子图：

- 在 Jupyter Notebook/Lab 中显示

如果在 Jupyter Notebook 或 JupyterLab 环境中使用 plotly，调用 `show()` 函数时，会直接在 Notebook 的输出单元格中嵌入交互式的图表。

- 在普通 Python 脚本中显示

如果在标准的 Python 脚本中运行 `show()`，且没有安装 Notebook 或类似工具，plotly 会启动本地 Web 服务器，并在系统默认的浏览器标签页中显示图表。

plotly 的 `show()` 函数实现逻辑在 plotly 库的内部，是一个通用的渲染函数，根据运行环境来决定如何显示图表。具体的显示方式由 `plotly.io` 模块的逻辑来处理，这个模块包含了处理图表渲染的不同机制。

在 plotly 中，`show()` 函数的逻辑会根据当前的执行环境来确定是内嵌显示图表还是启动外部浏览器。

实现逻辑的关键模块：

1. `plotly.io` 模块：
  - `plotly.io` 是一个用于处理图表渲染的模块。`show()` 函数最终会调用这个模块的函数来决定如何渲染图表。它根据当前运行的环境（例如 Jupyter Notebook 或标准的 Python 环境）来确定显示方式。
2. 环境检测与渲染方式：
  - 如果代码在 Jupyter 环境中执行，`plotly.io` 检测到这个环境后会使用内嵌的渲染方式（例如 `plotly_mimetype`）将图表直接输出到 Notebook。
  - 如果代码在标准 Python 脚本中运行，并且没有特殊的设置，`plotly.io` 会启动一个本地 Web 服务器来显示图表，并在默认浏览器中打开图表的页面。
3. `plotly.offline` 支持：
  - 在非交互式环境（例如终端或文本编辑器）中，可以使用 `plotly.offline` 来离线显示图表，通常也是通过浏览器打开一个本地文件来显示。

大致流程：

- 当 `show()` 被调用时, `plotly.io.show` 检查当前的运行环境。
- 如果是在 Jupyter Notebook 中, 它会使用 JavaScript 来内嵌图表到 Notebook 单元格中。
- 如果是在普通 Python 脚本中, 它会创建一个 HTML 文件, 并通过浏览器打开这个文件。

- 如果是在 Jupyter Notebook 中，它会使用 JavaScript 来内嵌图表到 Notebook 单元格中。
- 如果是在普通 Python 脚本中，它会创建一个 HTML 文件，并通过浏览器打开这个文件。

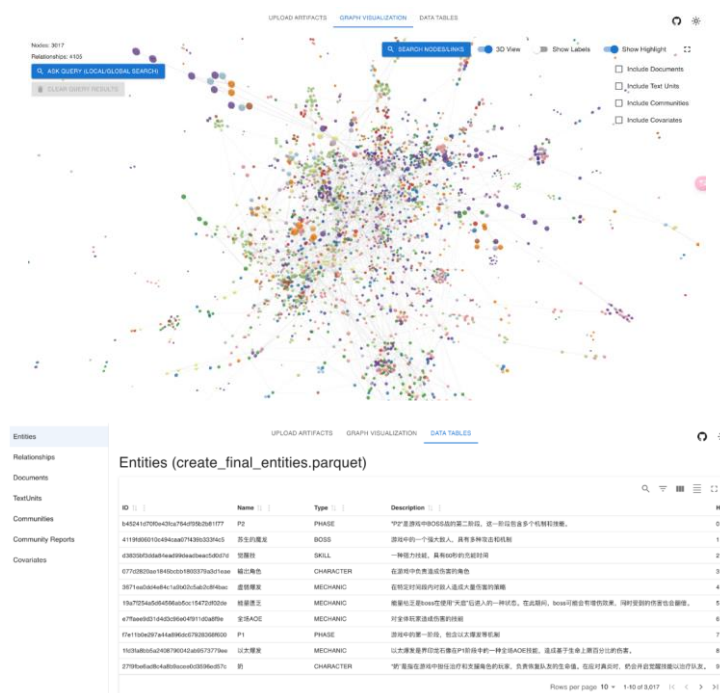
代码参考位置:

- `plotly.io.renderers.show` 是核心的渲染函数。
- 检测和处理环境的代码可能位于 `plotly.io.base_renderers.py` 文件内，它负责决定显示图表的策略。

## 2.2 在 javascript 中，使用 react-force-graph-2d 和 react-force-graph-3d

这个工程分成前后端，可以上传对应的 parque 文件，解析成 2d 或 3d 图形，同时提供

parque 文件对应的 table。通过查询模式，或者点击交互模式，找到对应节点信息。



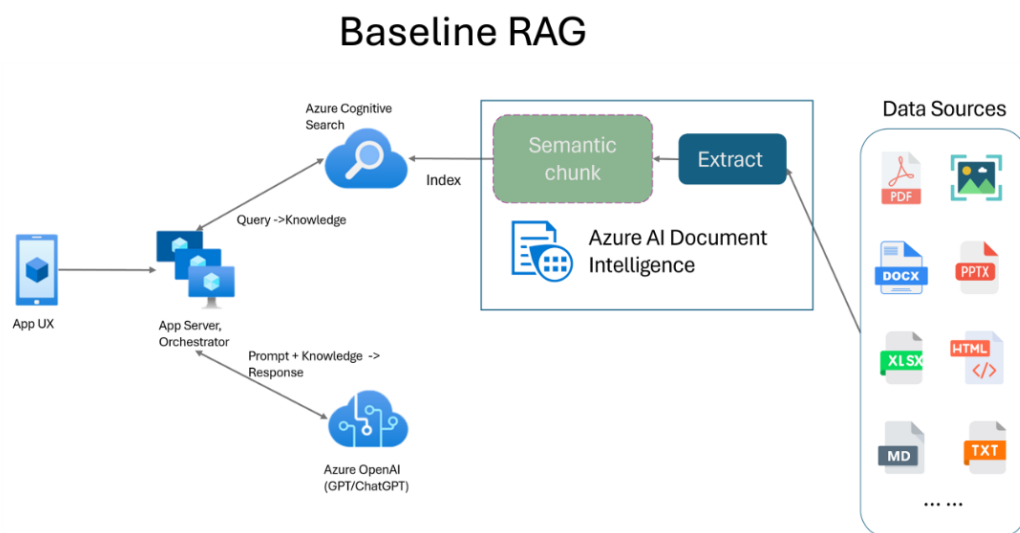
可以根据不同的需求，选择不同的工具和框架来展示图像数据，让业务人员能开发和业务人员弄对这个中间结果有一个整体的认识。

在最后的回答中，也通过展示这些信息，来反向追述到文件本身。

## 第四章：再讨论 RAG-GraphRAG with agentic rag

随着大语言模型的爆火，用户开始尝试通过 RAG 的方案来实现私域知识库的问题。如下是一个在 Microsoft Azure 平台上最简单常见的 baseline rag 端到端的处理架构图，主要分成两个部分：

- 离线文档部署：将各种文件格式通过不同的 chunk 策略来切割、索引。
- 在线文档：通过问题内容，查询索引获取到 top N 的 chunk 内容，再经过大语言模型的理解，回答相应的内容。



### 1. Baseline RAG 盲点和挑战

Baseline RAG 简单容易理解、效率高，能够快速验证，但由于流程简单，还存在如下四类问题：

#### 1.1. 总结类问题

- 总结一般要遍历所有 chunk，只总结 topN 的 chunk 答案不全面。
- 如果通过其他框架运行，可以使用 langchain 里面的 map-reduce chain 实现总结。

#### 1.2. 比较类问题

- 比较类问题需要查找出涉及不同内容的文档，但业务查询过程的打分机制，无法很好地将内容提取出来，导致回答出现问题。
- 如果比较的实体占比相差很大，比如用大象和蚂蚁的重量来表示内容占比，差距太大无法比较。

### 1.3. 隐式数据和关系型问题

- 隐式数据，是指不通过直接输入或显性方式提供，而是通过用户行为、操作或系统自动记录推断出来的数据。比如：房地产财报中业绩最好的公司，收入是多少？无法通过关键词或语义搜索获取答案。
- 通过对象与对象之间的关系提出的问题，比如：食物和人群之间的关系等。某个食物是否适合孕妇好回答，但是反过来问孕妇适合的食谱有哪些就难了。

### 1.4. 多头问题

- 问题本身是多个问题组成。
- 或者问题本身也包含了上述的三类问题。

## 2. 改进 RAG 的两个方向

### 2.1. 对数据进行处理-提高数据质量和索引

在数据预处理阶段，通过以问题为中心的方式来加工处理原数据，而不仅仅是按照原文本的方式。用户输出的文本内容是信息，如何才能把信息转化为知识？归纳法是很好的方式，它是由英国哲学家弗朗西斯·培根（Francis Bacon）提倡的一种科学方法论，通过观察具体的现象或事件，找出其中的规律或模式，并从中推导出普遍性的结论。

使用归纳法分析问题时，培根建议按照以下三个步骤进行：

- |                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"><li>1. 数据收集与整理：<ul style="list-style-type: none"><li>• 首先，通过观察和实验，系统地收集有关研究对象的所有信息和实例。这些信息被整理到三</li></ul></li></ol> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|



张表格中，以便进行后续分析。

2. 寻找共同点与差异：

- 在三张表格中寻找现象出现与消失的条件、不同强度下的表现等，通过比较与对照，逐步排除与现象无关的因素，找出可能的因果关系或本质特征。

3. 形成假设与验证：

- 根据前两个步骤得出的初步结论，提出可能的假设或理论，然后通过进一步的实验和观察进行验证和修正，最终形成可靠的科学结论。

GraphRAG 基本遵循这三个步骤实现对信息的处理：

- 实体就是信息的承载的主要载体，GraphRAG 通过抽取实体，解决比较类问题。
- 实体通过 leiden 算法，变成一个个社区，形成不同社区的总结。然后，可以通过 Global Search 逐个遍历社区得到整体内容，来应对总结类问题。
- 在 GraphRAG 里面，entity 和 entity 的关系也通过图的边来表示，抽取 claim 保存额外信息，能够解决隐式数据和关系型问题。

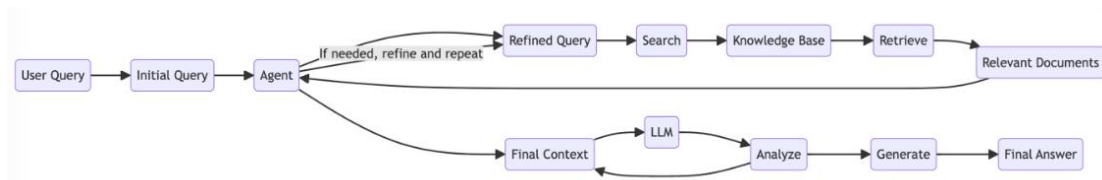
## 2.2. 对问题和答案进一步分析和处理

如果对于更加复杂的问题，仅仅对数据本身处理还不够对问题进行分拆、反思等预处理，提高检索到数据的概率，同时对结果进行评估，能够得出更准确的答案。

简单 RAG 处理流程如下：



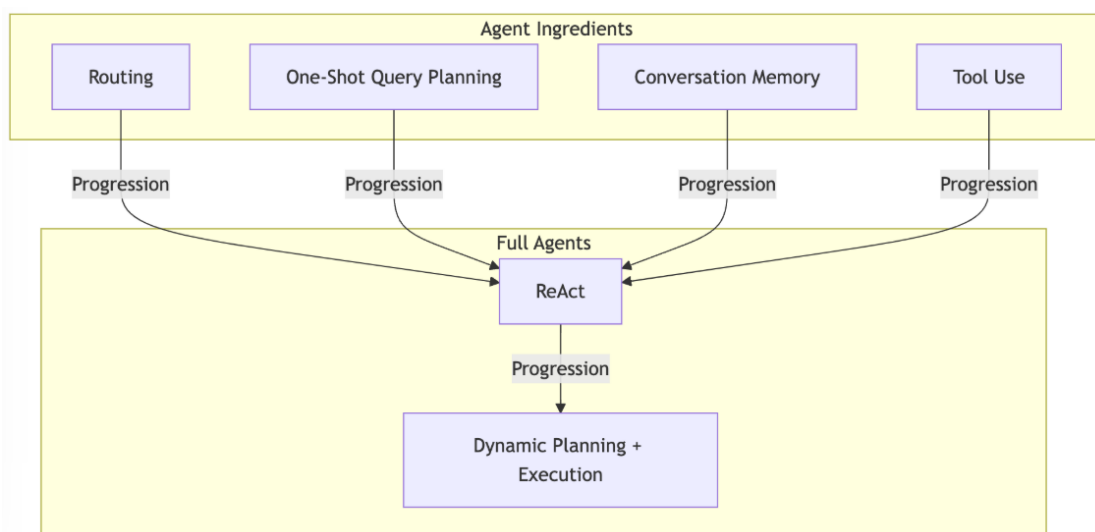
agentic RAG 处理流程如下：



一个 RAG 的 agent 通常由四个基本功能组成：

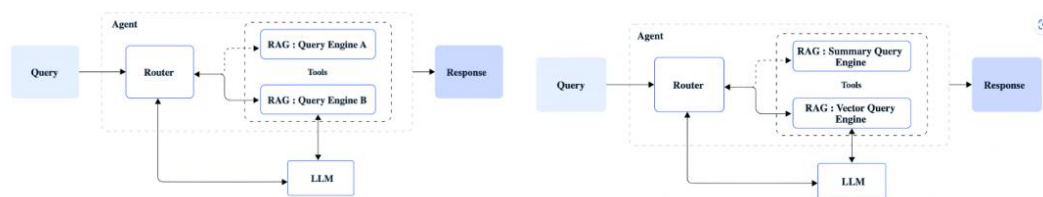
- 路由
- 查询规划
- 对话记忆管理

## 4. 工具的使用



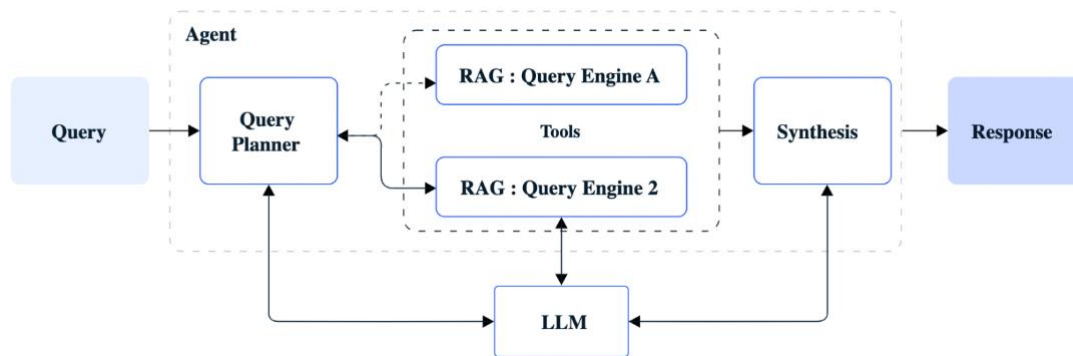
### 2.2.1 Routing (路由): 处理和分配任务或请求的能力

一种路由代理使用大型语言模型 (LLM)，来确定要选择哪个下游 RAG 管道。此过程构成代理推理，通过分析 LLM 输入查询，选择最合适的 RAG 管道做出明智的决策，是代理推理的基本和简单形式。另一种路由涉及在摘要和问答 RAG 管道之间进行选择。代理评估输入查询，以决定是将其定向到摘要查询引擎还是矢量查询引擎，两者都配置为工具。



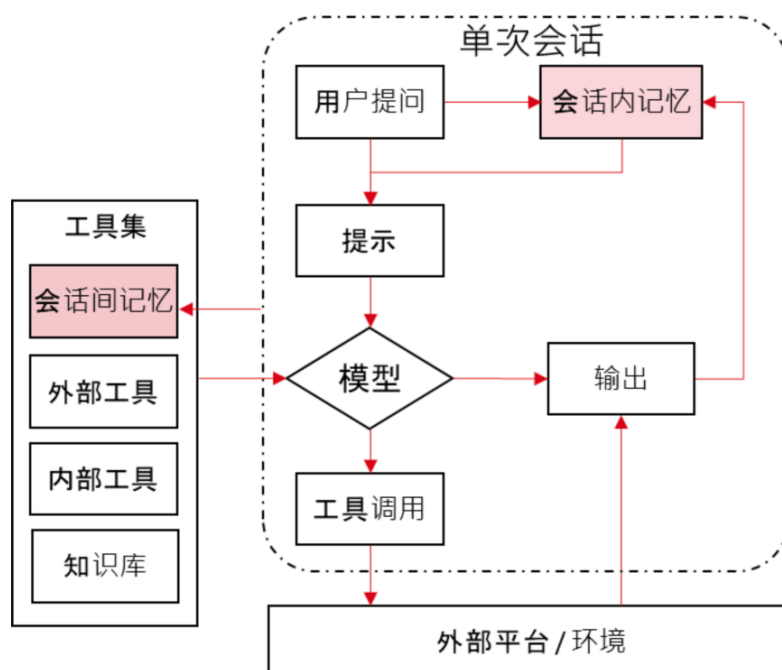
### 2.2.2 One-Shot Query Planning (一次性查询规划): 一次性处理查询和规划响应的能力

查询规划代理将复杂查询划分为可并行化的子查询,每个子查询都可以根据不同的数据源在各种 RAG 管道中执行。然后,来自这些管道的响应再合并到最终响应中。



### 2.2.3 Conversation Memory (对话记忆): 记住和管理对话历史的能力

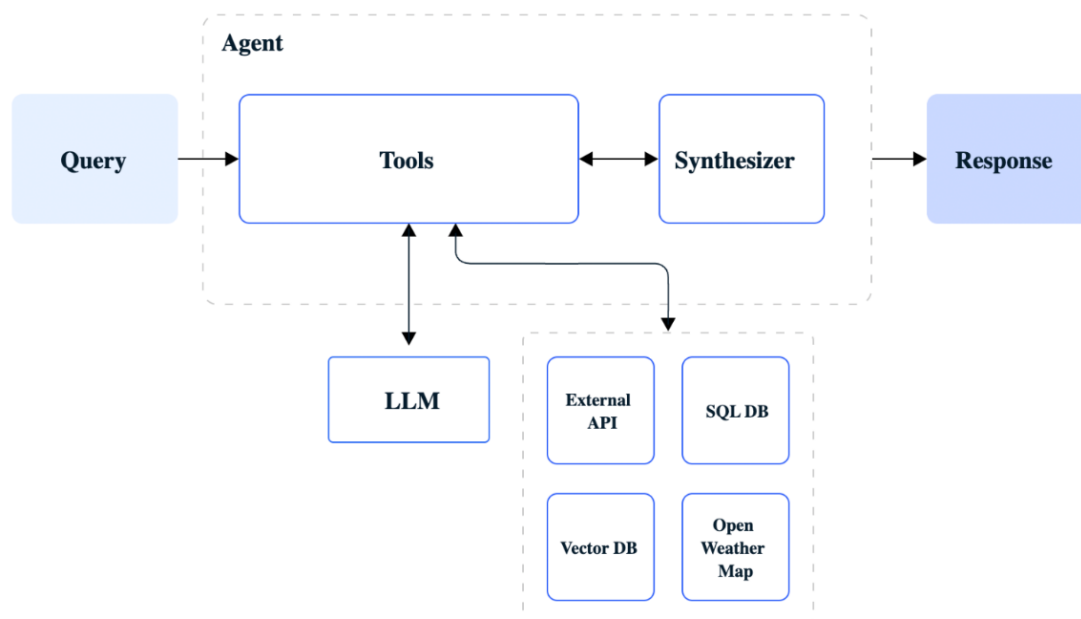
在 Agent 类应用中，记忆可以记住事实和偏好，如问题答案、工具调用结果、格式偏好、语言偏好、表达偏好、图表绘制偏好、任务下工具选择偏好等，从而让 Agent 应用提升回答准确率、稳定性以及速度。基于存储方式，可以把记忆分为短期记忆和长期记忆。从会话角度，记忆还可被分为：会话内记忆和会话间的记忆。



### 2.2.4 Tool Use (工具使用): 使用各种工具来完成任务的能力

在典型 RAG 中，需要提交查询来检索在语义上与查询匹配的文档。但是某些情况下，需要将来自外部源（例如 API、SQL 数据库或具有 API 接口的应用程序）的其他数据用作上下文，在输入查询被处理之前增强输入查询 LLM。在这种情况下，代理也可以使用 RAG 规

范。

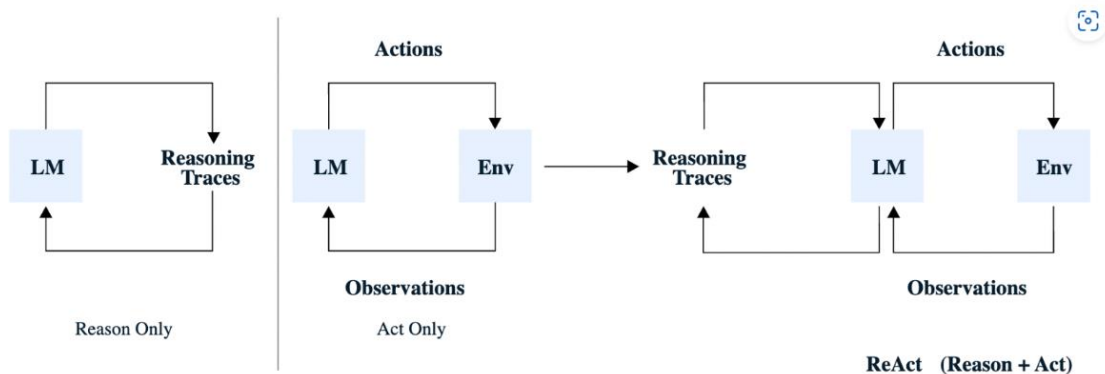


当发展成为高级 agent 是，处理流程还将拥有以下能力：

### 2.2.5 ReAct: 具备反应能力，根据输入动态调整行为

升级到更高层次涉及合并推理和操作，这些推理和操作是在复杂查询上迭代执行的。从本质上讲，这包括将路由、查询规划和工具使用组合到一个实体中。ReAct 代理能够在保持状态（在内存中）的同时处理顺序多部分查询。该过程包括以下步骤：

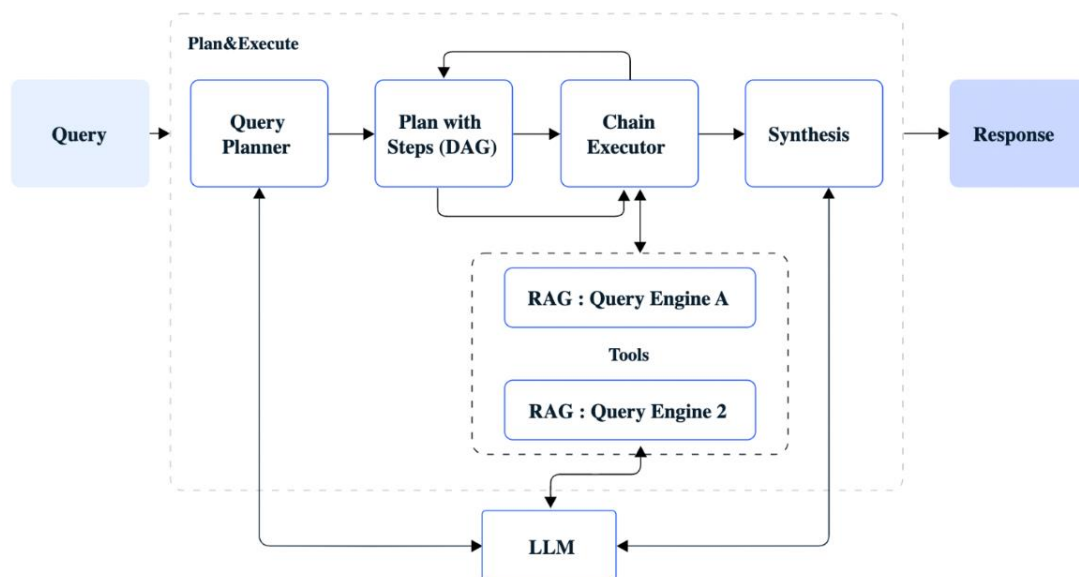
- 在收到用户输入查询后，代理会确定要使用的适当工具（如有必要），并收集该工具的必要输入。
- 使用必要的输入调用该工具，并存储其输出。
- 代理接收工具的历史记录，包括输入和输出，并根据此信息确定后续操作过程。
- 此过程将循环进行，直到代理完成任务并响应用户。



## 2.2.6 Dynamic Planning + Execution (动态规划与执行): 具备动态规划和执行

### 复杂任务的能力

ReAct 是目前被广泛采用的代理工具。然而，随着对处理更复杂用户意图的需求日益增长，对代理的要求也在不断提高。特别是在代理部署到生产环境的数量增加时，对于提升其可靠性、可观察性、并行处理能力、控制力以及实现关注点分离的需求变得尤为迫切。从本质上讲，需要长期规划、执行洞察力、效率优化和减少延迟。



Agentic RAG 能够实现对问题的分解和对中间信息的反思，相应等，来解决多头问题。

## 3. agentic RAG 方案

构建代理 RAG 需要特定的框架和工具，来促进多个代理的创建和协调。虽然从零开始构建

这样的系统相对复杂，但已有一些解决方案可以帮助简化实施过程。

### **3.1. LlamaIndex**

LlamaIndex 是构建代理系统的坚实基础，提供一整套功能。它使开发人员能够创建文档代理，监督代理交互，并实施高级推理机制，例如思维链。该框架提供了许多预构建的工具，促进与各种数据源的交互，包括流行的搜索引擎（如 Google）和存储库（如 Wikipedia）。它与各种数据库无缝集成，包括 SQL 和向量数据库，并支持通过 Python REPL 执行代码。LlamaIndex 的链功能使不同工具的无缝链接成为可能，从而 LLMs 促进了复杂工作流程的创建。

此外，它的记忆组件有助于跟踪代理操作和对话历史，从而促进上下文感知决策。包含针对特定用例（例如聊天机器人和问答系统）量身定制的专用工具包，进一步增强了其实用性。但是，要充分利用其潜力，可能需要熟练掌握编码和理解底层架构。

### **3.2. langchain**

与 LlamaIndex 一样，LangChain 提供了一个全面的工具包，用于构建基于代理的系统并协调它们之间的交互。其一系列工具与 LangChain 生态系统内的外部资源无缝集成，使代理能够访问广泛的功能，包括搜索、数据库管理和代码执行。

LangChain 的可组合性功能使开发人员能够结合不同的数据结构和查询引擎，从而促进创建能够访问和操作来自各种来源的信息的复杂代理。其灵活的框架可以很容易地进行调整，以适应代理 RAG 实现中固有的复杂性。

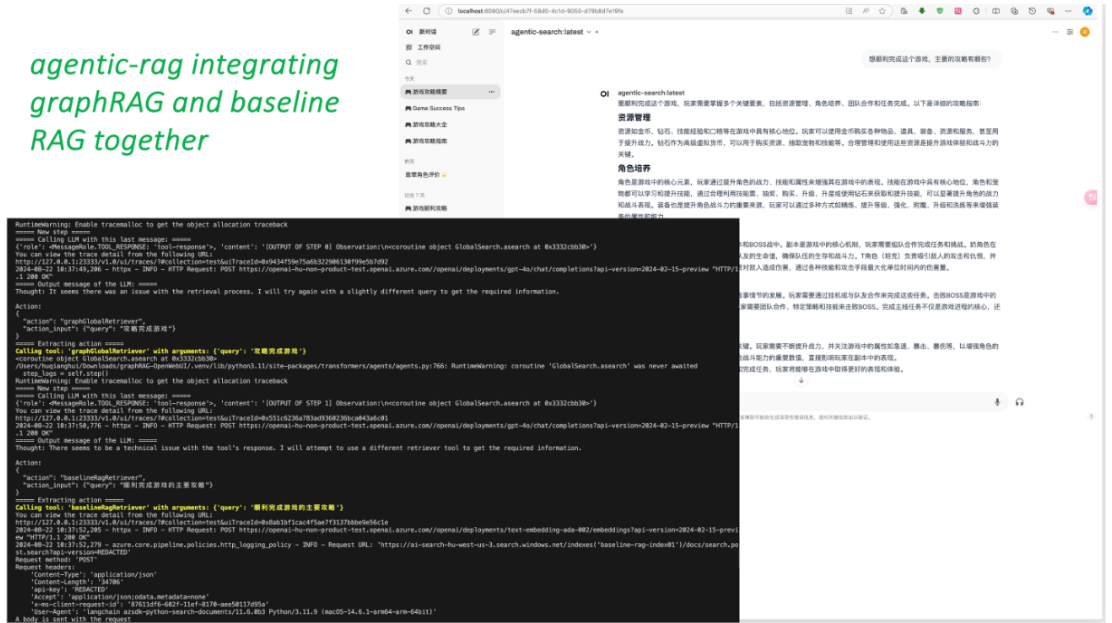
### **3.3. transform agent**

Hugging Face (HF) 推出了 Transformers Agents 框架，旨在通过简单性实现清晰度。该框架将抽象减少到最低限度，通过简单的错误日志和可访问的属性，可以轻松检查所发生的情况，更加清晰地理解系统状态。

框架的另一个核心理念是模块化。Hugging Face 更倾向于提供构建块，而不是完整、复杂的功能集。这种方式让您可以自由选择最适合您项目的任何构建块。例如，由于任何代理系统都只是由 LLM（大型语言模型）引擎驱动的车辆，Hugging Face 决定在概念上将两者分开。这种设计使您可以从任何底层创建任何代理 LLM 类型，提供了极大的灵活性和选择自由。

#### 4. 通过 agentic 整合 graphRAG 和 baseline RAG

很多企业在 baseline RAG 进行了大量投资，并且许多系统已经在线运行。那么，如何实现 GraphRAG 与 Baseline RAG 的共存呢？答案就是使用 agentic RAG 方案。通过一个简单的路由功能，我们可以将那些不适合 Baseline RAG 而更适合 GraphRAG 处理的问题，直接路由到 GraphRAG 的处理流程中。这样，两种 RAG 就可以协同工作，发挥各自的优势。







## 第五章：GraphRAG 的最佳实践

### 1. 数据的 chunk 策略

Graphrag 采取的是固定大小的 chunk 策略，在 setting.xml 里面可以设定。

这种 chunk 策略的优点是：

- 1) 实现简单：不需要复杂的文本分析，只需要根据字符或词的计数进行分隔。
- 2) 一致性：每个块都有固定大小，使得计算和存储易于管理。在计算与 LLM 交互过程中来估算窗口大小也很方便。
- 3) 通用性强：适用于几乎所有的文本类型，无需特定领域的语义分析和业务知识。

优点的同时也是缺点，很多情况使得不同块在语义上割裂，冗余太低导致搜索引擎不管是关键字搜索，或者语义搜索的向量搜索在一部分情况下没有办法把连续块召回，从而 LLM 没有办法很好的理解知识的全貌。所以它主要使用在简单的任务，或者文本本身对语义切割不敏感的场景。

很显然在 graphrag 的构建过程中，涉及到很多文档本身是语义很敏感的，比如在文档中有比较长段的论述和有长表格等，这种情况下需要使用语义切割和上下文补足，才能让抽取更好的知识图谱，同时在回答过程中招呼更完整的内容。我们通过这种方式切割好，保存一个个独立的文件，通过设置 graphrag 的块大于我们的块大小，这样就能保证一个个文件就是一个单独的块。从而在 graphrag 里面实现更好的效果。

在这里主要使用到了两个技术：

- 1) 语义切割 (semantic chunk)
- 2) 基于上下文的 RAG (Contextual RAG)

通过语义切割保证块的完整性，通过 Contextual RAG 可以实现对 chunk 的内容进行意义

补足。

## 2. 提示词调优 (prompt-tuning)

Graphrag 提供了默认提示词。它是开始使用 GraphRAG 系统的最简单方法。它旨在以最少的配置开箱即用。但是它没有给定特定领域，语言默认页是英语，实体类型也是最常见的地理，事件等少量的几个类型。在实际使用过程，往往不能满足需求。

### 2.1 自动调整 (Auto Tuning)

GraphRAG 提供了 auto-tuning 的功能，只需要给定几个文件和配置策略，它就可以针对给定的文件内容来重新生成一版本偏向指定文本内容的提示词，这样能够很好的把文本中的知识，抽取成更丰富的图谱。

命令实例：

```
python -m graphrag prompt-tune --root /path/to/project --config
/path/to/settings.yaml --domain "games" \
--method random --limit 10 --language Chinese--max-tokens 2048 --chunk-
size 256 --min-examples-required 3 \
--no-entity-types --output /path/to/output
```

得到对应的提示词之后，便可以使用对应的 prompt 来生成知识图谱。

```
python -m graphrag.prompt_tune \
--config settings.yaml \
--root ./ragtest \

```

```
--domain games \

--selection-method random \

--limit 10 \

--language Chinese \

--max-tokens 2048 \

--chunk-size 512 \

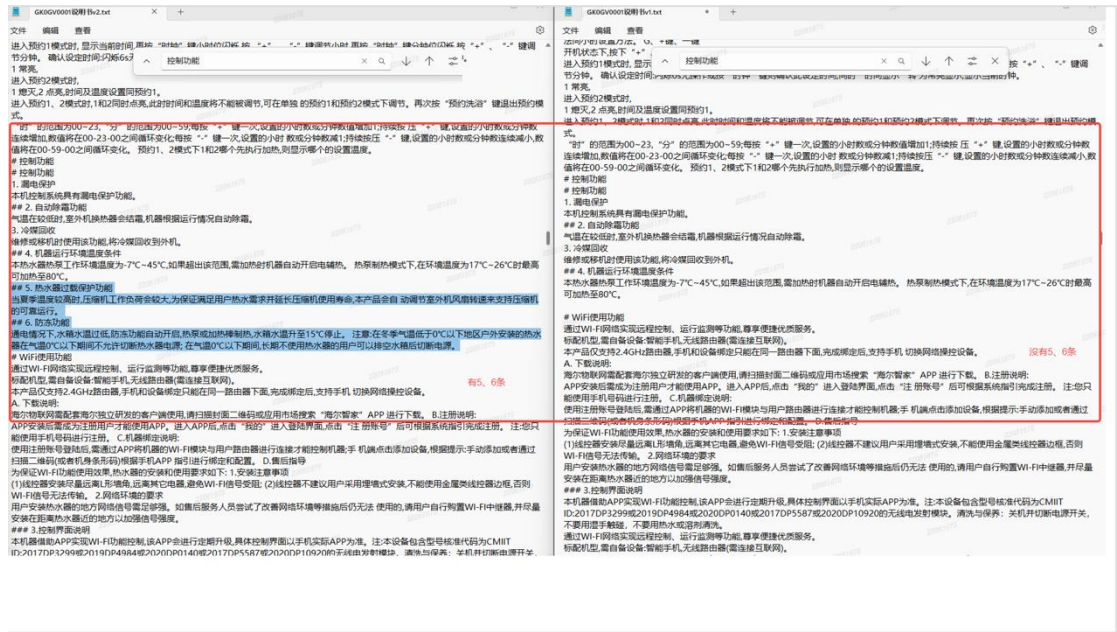
--no-entity-types \

--output ./ragtest/prompts/tuned-prompts
```

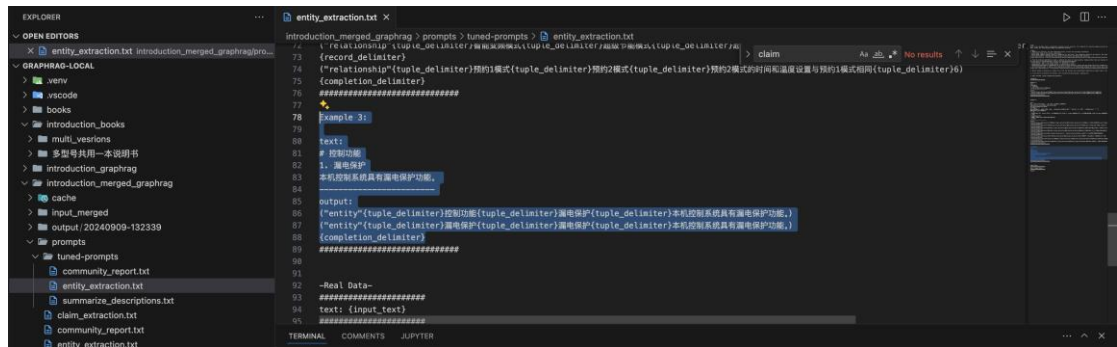
## 2.2 手动调优 (Manual Tuning )

手动调优是一种高级用例。是使用 Auto Tuning 功能的基础之上，发现生成的知识图谱任然没有达到自己的预期，或者一些特定的实体还是没有出现等情况，就需要进一步的手动调优。

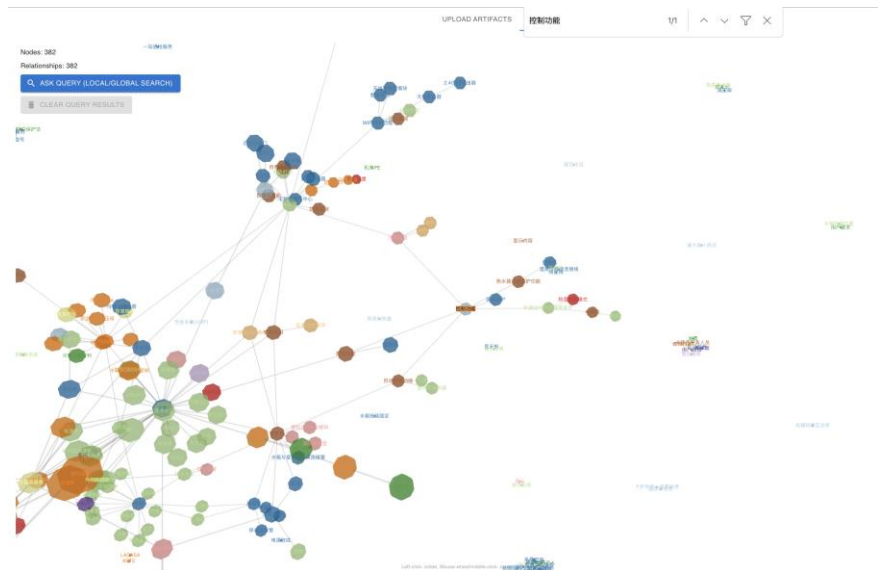
在一个客户例子中，用户手册的更新，不同版本的功能不同，希望能够比较好的完成所有的功能点都能出现在知识图谱中。



这种情况下，就通过手动的指定提示词来实现：



最后的知识图谱的可以看到对应功能的六个功能。



### 3. 与图数据库的整合

现有的 graphrag 主要集中在图的生成，以及查询优化上，默认的查询和计算都是基于文件和内存的。但是知识图谱进一步变大变复杂，或者要推广到生成环境，图数据库作为知识图谱数据库必不可少的中间件。

在前面章节中我们已经介绍了各种图形数据，以及它们之前的差异性。这里主要介绍三种图形数据库的解决方案。

#### 1) cosmosDB 整合方案

cosmosDB 作为 azure 云平台上的原生的图数据库，所以在实现上我最初的选择还是它。但是也确实发现了一些问题，比如说：它的图数据库不支持向量计算，所以需要联合一个其他的服务，这里我选择了 azure AI search，让它完成向量搜索，然后通过图的语法来构建上下文，实现 graphrag 的查询。

代码实现的链接如下：

<https://github.com/huqianghui/graphrag-cosmosdb-AI-search-intergration>

在这里主要实现了三个 API。

1. import: 把 graphrag 生成的 parque 文件导入到 Azure AI search 和 Azure CosmosDB 中

```

def create_search_index(index_name, index_client, client):
 print(f"Ensuring search index '{index_name}' exists")
 if index_name not in index_client.list_index_names():

 index = SearchIndex(
 name=index_name,
 cors_options=CorsOptions(allowed_origins=["*"], max_age_in_seconds=600),
 fields=[
 SimpleField(name="id", type=SearchFieldType.String, key=True, searchable=True, filterable=True, sortable=True, facetable=True),
 SearchableField(name="name", type=SearchFieldType.String, analyzer_name="zh-hans.microsoft"),
 SearchableField(name="type", type=SearchFieldType.String, analyzer_name="zh-hans.microsoft"),
 SearchableField(name="description", type=SearchFieldType.String, analyzer_name="zh-hans.microsoft"),
 SimpleField(name="human_readable_id", type=SearchFieldType.String, searchable=True, filterable=True, sortable=True, facetable=True),
 SearchableField(name="description_embedding",
 type=SearchFieldType.Collection(SearchFieldType.Single),
 hidden=True,
 searchable=True,
 filterable=False,
 sortable=False,
 facetable=False,
 vector_search_dimensions=1536,
 vector_search_profile_name="azureOpenAI512svProfile"),
],
 semantic_search=SemanticSearch(
 configuration=[
 SemanticConfiguration(
 name="default",
 prioritized_fields=SemanticPrioritizedFields(
 title_field=SemanticField(field_name="name"),
 content_fields=[
 SemanticField(field_name="description")
],
 keywords_fields=[
 SemanticField(field_name="type")
]
)
)
]
),
 vector_search=VectorSearch(
 algorithm=
 HnswAlgorithmConfiguration(
 name="myHnsw")
 ,
 profiles=[
 VectorSearchProfile(
 name="azureOpenAI512svProfile",
 retriever=

```

Home > Azure AI services | AI Search > ai-search-hw-west-us-3 | indexes > graphrag\_entity\_index

Save Discard Refresh Create demo app Edit JSON Delete

Documents 382 Total storage 9.66 MB Vector index size 2.27 MB Max storage 25 GB

Search explorer Fields CORS Scoring profiles Semantic configurations Vector profiles

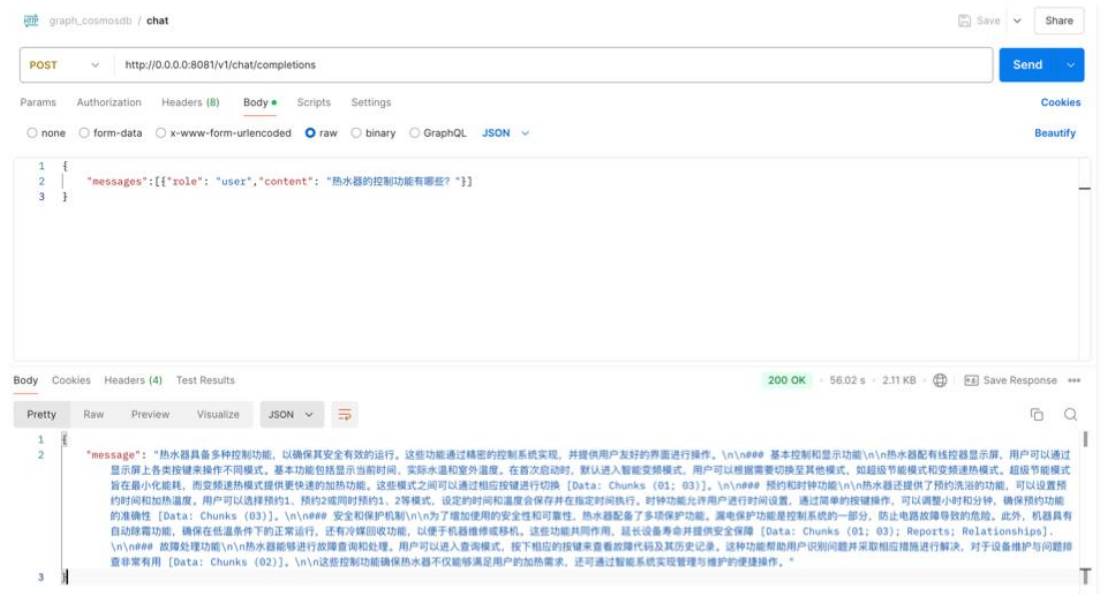
+ Add field + Add subfield Delete Autocomplete settings

Search field names

Field name	Type	Retrievable	Filterable	Sortable	Facetable	Searchable	Analyzer	Dimensions
id	String	<input checked="" type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>		
name	String	<input checked="" type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input checked="" type="checkbox"/>	Chinese	
type	String	<input checked="" type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input checked="" type="checkbox"/>	Chinese	
description	String	<input checked="" type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input checked="" type="checkbox"/>	Chinese	
human_readable_id	String	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>		
description_embedding	SingleCollection	<input checked="" type="checkbox"/>				<input checked="" type="checkbox"/>		

1536

2. localSearch: 通过 Azure AI search 查找到 top K 的 entity, 根据这些 entity 来构建 context, 实现 localSearch 的查询



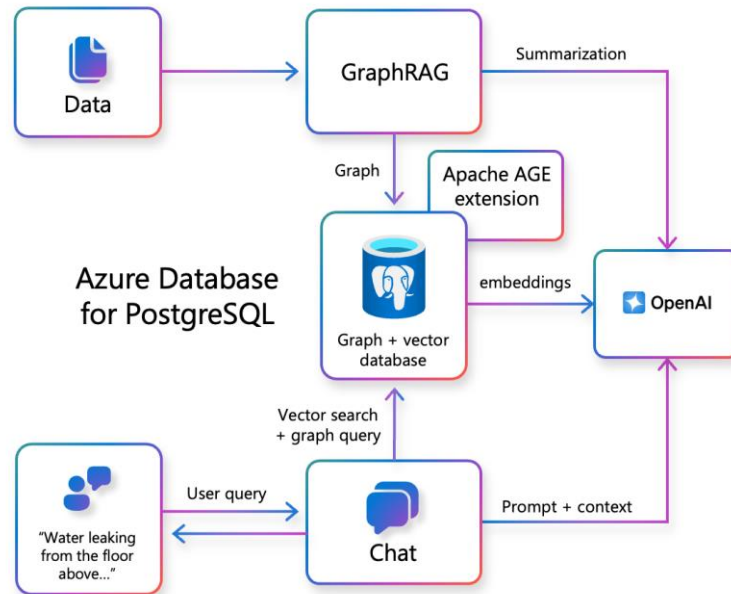
### 3. globalSeach：在 cosmosDB 查找到相应的所有社区，通过 map reduce

查找。在 map 过程中使用多个节点轮询，避免出现 429 的问题。

更多细节请参考对应的代码库和说明文档。

## 2) PostgreSQL 解决方案

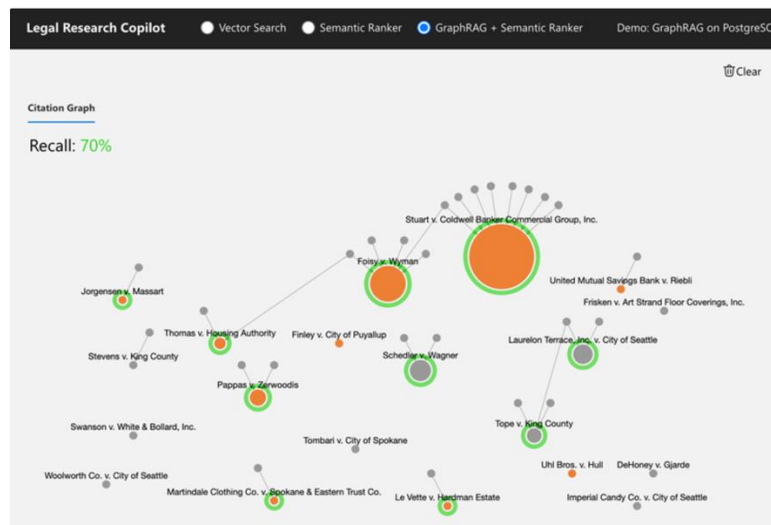
在 2024 年 11 月的微软的 ignite 大会上，给出了适用于 Azure Database for PostgreSQL 的 GraphRAG 解决方案。如下图展示：



对应的代码实现库地址：

<https://github.com/Azure-Samples/graphrag-legalcases-postgres>

它使用一个法律文档作为例子，查看 GraphRAG 信息检索管道的搜索结果 UI：



它使用了 Postgre SQL 数据的一个 Apache AGE 扩展实现图数据的查询，同时 Postgre SQL 自身也支持向量搜索，所以可以在在这一个中间件数据库中完成向量排序和图查询操作。同时通过利用语义排名程序的强大功能，GraphRAG 和 Azure Database for PostgreSQL 企业可以确保其生成式 AI 投资取得成功。随着这项技术的不断发展，它有望将生产力提升到新的水平，并推动各个领域的 GenAI 创新。



## 4. 多模态 graphrag

多模态 RAG 是一种结合图像和文字两种模态进行检索增强生成的技术，它通过整合多模态的输入（通常是文本和图像），提高对复杂场景的理解与生成能力。这种方法尤其适合需要跨模态信息融合的任务，如视觉问答、图文生成、图片内容解读和多模态搜索引擎等。在 baseline RAG 中已经进行较多的探索 and 实现，但是某人的 graphrag 目前输入输出还是文字。但是如果按照之前的设计，它作为 baseline rag 的补充，baseline rag 能有的它也必须补齐，所以也实现了一个支持图片搜索的多模态的 graphrag。

目前功能还比较简单，就是通过 gpt-4o 这种本身就支持图片的模型来支持，通过集成原始的 graphrag 搜索类，实现了一个 MultiModelLocalSearch。

具体代码参考链接：

<https://github.com/huqianghui/graphRAG-OpenWeb-agentic-backend/blob/main/customizedMultiModelLocalSearch.py>

\*\*这里代码库中把 agentic rag，多模态 graphrag，包括 graphrag 中元数据过滤都有一些尝试。\*\*

如图展示一个多模态 graphrag 的搜索例子：



这个图片对用的内容有哪些？



graphrag-local-search:latest

这张图片展示的是一个传统的书房或教室环境，可能与鲁迅的作品《从百草园到三昧书屋》中的三昧书屋有关。三昧书屋是鲁迅童年时光的学习环境，代表了他成长过程中的重要场景 [Data: Reports (81); Entities (1)]。

书房环境

- 布局与装饰: 图片中可以看到传统的木制桌椅，墙上挂着字画，营造出一种古朴的学习氛围。这种环境细节描写可以暗示人物的性格或生活状态 [Data: Entities (728)]。
- 文化氛围: 书房的布置和装饰反映了对传统文化的尊重和热爱，可能与鲁迅作品中对学习环境的描绘相呼应 [Data: Reports (81)]。

相关背景

- 鲁迅与三昧书屋: 鲁迅在《从百草园到三昧书屋》中描绘了三昧书屋，作为他童年学习的场所，象征着自然与学习环境的结合 [Data: Reports (81); Relationships (4)]。

这张图片通过展示传统书房的细节，传达了浓厚的文化氛围和历史感，可能与鲁迅的文学世界有着密切的联系。

< 4/4 > [edit] [delete] [share] [comment] [refresh]

+ 输入消息



## 第六章：未来展望

### 1. 通过知识图谱进一步推理出新的知识

随着人工智能技术的飞速发展，大语言模型在自然语言处理领域展现出了前所未有的能力。利用大语言模型，从非结构化文档中提取高质量的知识图谱，已成为信息抽取和知识管理的重要手段。知识图谱以结构化的形式呈现实体和关系，极大地提高了信息的可用性和理解度。然而，如何进一步利用这些结构化的知识图谱，借助先进的推理模型，如 OpenAI 的 O1 模型，来拓展和推理出更多有效且泛化的信息，成为当前研究的热点。首先，大语言模型具备强大的自然语言理解和生成能力，能够从海量的非结构化文本中自动提取出实体、属性和关系，构建出高质量的知识图谱。这种知识图谱不仅涵盖了丰富的领域知识，而且在数据质量和一致性方面有了显著提升。传统的知识获取方式往往依赖于人工整理，效率低下且容易出错，而大语言模型的引入，极大地缩短了知识获取的周期，提高了准确性和完整性。

然而，仅有高质量的知识图谱还远远不够。为了充分挖掘其中蕴含的潜在信息，我们需要借助先进的推理模型。OpenAI 的 O1 模型作为当前最前沿的推理模型之一，具备强大的推理和泛化能力。通过将知识图谱输入 O1 模型，我们可以在已有知识的基础上，推理出更多隐含的关系和信息。例如，基于已知的生物学知识图谱，O1 模型可以预测出某些尚未被发现的生物互动关系，为科学研究提供新的方向。

O1 它能够处理复杂的逻辑关系，不仅限于简单的实体关联，能够根据输入的数据不断优化自身的推理路径，提高推理的准确性，能够在不同领域的知识图谱上进行有效推理和信息挖掘。

## 2. 结合和使用企业内部的知识图谱，挖掘更多有价值的信息

GraphRAG (Graph Retrieval-Augmented Generation) 作为一种先进的技术手段，可自动生成知识图谱，为企业的数据管理和智能分析提供了新的可能。然而，很多企业已经拥有了自己的知识图谱体系，如何将通过 GraphRAG 生成的知识图谱与企业部门现有的知识图谱实现联动，互相补充，形成一个更全面、更实时的知识图谱，是一个很有趣的主题。

### 知识图谱联动与互补的挑战

- 数据异构性:** 企业现有知识图谱和 GraphRAG 生成的知识图谱可能来自不同的数据源，数据格式、结构和语义可能存在差异，给融合带来困难。
- 语义一致性:** 不同知识图谱中的概念和关系可能存在同义词、近义词或多义词等问题，需要解决语义上的差异，确保融合后的知识图谱语义一致。
- 数据冗余与冲突:** 在融合过程中，可能出现重复的数据或矛盾的信息，需要有效的方法来识别和处理。